UNIVERSIDAD DE LA GUAJIRA

Facultad de Ingenierías · Ingeniería de Sistemas

Parcial 3 · Ingeniería de Software

Equipo de calidad y mantenimiento de *EnvíaYa*

Estudiante	Carlos David Sierra
Código estudiantil	0182410125
Asignatura	Ingeniería de Software I (273365) · Grupo B1 · VI semestre
Docente	Ing. Robert Damián Quintero Laverde
Periodo	2026-2 · Tercer corte
Fecha	11 de noviembre de 2026
Repositorio	https://github.com/calo159/parcialll
Lenguaje	JavaScript (Node.js) + Jest — traducción fiel de `envios.py`, defectos incluidos

Mis parámetros

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
4	1	0	1	2	5	50	3	10	20 %	50 %	15 %	$2.000	0

D = 1 impar → tramo 1: peso < 3 · E = 2 par → fin de semana antes de descuentos · B = 1 impar → se aplican ambos descuentos (cliente y luego cupón) · F = 5 impar → cupón inexistente es error · CAMBIO = 0 → zona insular

Parte 1 · Requisitos y criterios de aceptación

Parte 1 · Requisitos — Especificación personalizada

Estudiante: Carlos David Sierra · **Código:** 0182410125

Mis parámetros

Últimos 6 dígitos del código: **4 1 0 1 2 5**

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
4	1	0	1	2	5	50 kg	3 kg	10 kg	20 %	50 %	15 %	$2.000	0

Cálculo de cada parámetro:

Parámetro	Fórmula	Cálculo	Valor
PMAX	30 + 5·A	30 + 5·4	**50 kg**
L1	2 + (B mod 3)	2 + (1 mod 3) = 2 + 1	**3 kg**
L2	10 + C	10 + 0	**10 kg**
RR	10 + 5·(E mod 4)	10 + 5·(2 mod 4) = 10 + 10	**20 %**
RN	40 + 5·(F mod 3)	40 + 5·(5 mod 3) = 40 + 10	**50 %**
V	10 + 5·(A mod 3)	10 + 5·(4 mod 3) = 10 + 5	**15 %**
S	2000 + 500·(C mod 4)	2000 + 500·0	**$2.000**
CAMBIO	(A + F) mod 3	(4 + 5) mod 3 = 9 mod 3	**0**

Reglas por paridad:

Dígito	Valor	Paridad	Regla que me toca
D	1	impar	Tramo 1 **no incluye** el límite: peso **< 3**
E	2	par	El recargo de fin de semana se suma **antes** de los descuentos
B	1	impar	Cliente y cupón: se aplican **ambos**, primero el del cliente y luego el cupón
F	5	impar	Un cupón que no existe es **error**

Solicitud de cambio (Parte 8): **0 → zona insular (San Andrés), recargo del 70 %, sin cupón.**

CONFIG del código heredado

```
const CONFIG = {
  L1: 3,
  L2: 10,
  RECARGO_RURAL: 0.2,
  RECARGO_NACIONAL: 0.5,
  DESC_VIP: 0.15,
```

```
  RECARGO_FINDE: 2000,
};
```

Especificación R1–R8 (con mis valores)

Entradas: `peso` en kg (con decimales); `zona` ∈ {urbana, rural, nacional}; `cliente` ∈ {normal, frecuente, vip}; `dia` de 1 a 7 (6 = sábado, 7 = domingo); `cupon` opcional.

ID	Regla
R1	El peso debe cumplir **0 < peso ≤ 50**. Si no, se lanza el error **"peso inválido"**.
R2	Si la zona no es urbana, rural o nacional, o el día no está entre 1 y 7, se lanza un error.
R3	Tarifa base. **Tramo 1: peso < 3 kg → $6.000.** Tramo 2: 3 ≤ peso ≤ 10 → $9.000. Más de 10 kg: $9.000 + $800 por cada kg sobre 10, de forma proporcional.
R4	Recargo por zona: rural **+20 %**, nacional **+50 %**, urbana sin recargo.
R5	Sábado (6) y domingo (7): **+$2.000**, sumados **antes** de los descuentos.
R6	Descuento de cliente: frecuente 5 %, VIP **15 %**. Cupón `ENVIO10` : 10 %. Si hay ambos se aplican **los dos**: primero el del cliente y luego el cupón sobre el resultado.
R7	Un cupón que no existe **lanza un error**.
R8	El costo nunca es menor que $5.000. El resultado se redondea al múltiplo de $100 más cercano; los $50 redondean hacia arriba.

Orden de cálculo: validar (R1, R2, R7) → tarifa base (R3) → recargo de zona (R4) → recargo de fin de semana (R5) → descuento de cliente → cupón (R6) → mínimo de $5.000 → redondeo (R8).

Supuesto documentado: la especificación no dice qué hacer con un tipo de cliente desconocido; se conserva el comportamiento del código heredado (se trata como cliente normal, sin descuento).

Ejemplo calculado a mano con mis parámetros

Entradas: peso = 4 kg · zona = rural · cliente = vip · día = 6 (sábado) · cupón = ENVIO10

1. Tramo: D = 1 impar → tramo 1 es peso < 3. 4 no es < 3 y 4 ≤ 10 → tramo 2 → **$9.000**
2. Zona rural +20 % → 9.000 × 1,20 = **$10.800**
3. Sábado y E = 2 par → recargo antes de descuentos → 10.800 + 2.000 = **$12.800**
4. B = 1 impar → ambos: VIP 15 % → 12.800 × 0,85 = $10.880; luego cupón 10 % → 10.880 × 0,90 = **$9.792**
5. Mayor que $5.000. Redondeo al múltiplo de $100 más cercano → **resultado esperado: $9.800**

Parte 1 · Historias de usuario y matriz de trazabilidad

HU-01 · Validar los datos del envío (R1, R2, R7)

Como asesor de EnvíaYa **quiero** que el sistema rechace datos de envío inválidos **para** no cotizar envíos imposibles ni aplicar cupones falsos.

#	Dado	Cuando	Entonces
CA-1.1	un envío de 0 kg	calculo el costo	el sistema lanza el error "peso inválido"

#	Dado	Cuando	Entonces
CA-1.2	un envío de 50,01 kg	calculo el costo	el sistema lanza el error "peso inválido"
CA-1.3	un envío de 50 kg	calculo el costo	el sistema lo acepta y cotiza $41.000
CA-1.4	la zona "marte"	calculo el costo	el sistema lanza error de zona inválida
CA-1.5	el día 0 u 8	calculo el costo	el sistema lanza error de día inválido
CA-1.6	el cupón "GRATIS" (no existe)	calculo el costo	el sistema lanza error de cupón inválido

HU-02 · Calcular la tarifa con recargos (R3, R4, R5)

Como cliente de EnvíaYa **quiero** que el precio dependa del peso, la zona y el día **para** pagar un precio justo por mi envío.

#	Dado	Cuando	Entonces
CA-2.1	un paquete de 2,99 kg urbano un lunes	calculo el costo	el costo es $6.000 (tramo 1)
CA-2.2	un paquete de exactamente 3 kg urbano un lunes	calculo el costo	el costo es $9.000 (el límite 3 no pertenece al tramo 1)
CA-2.3	un paquete de 12 kg urbano un lunes	calculo el costo	el costo es 9.000 + 2 × 800 = $10.600
CA-2.4	un paquete de 5 kg a zona rural	calculo el costo	se cobra +20 %: $10.800
CA-2.5	un paquete de 5 kg a zona nacional	calculo el costo	se cobra +50 %: $13.500
CA-2.6	un paquete de 5 kg urbano un sábado o domingo	calculo el costo	se suman $2.000 antes de descuentos: $11.000

HU-03 · Aplicar descuentos, mínimo y redondeo (R6, R7, R8)

Como gerente comercial de EnvíaYa **quiero** que los descuentos de cliente y de cupón se apliquen en orden y que el precio final sea redondo **para** fidelizar clientes sin perder dinero.

#	Dado	Cuando	Entonces
CA-3.1	un cliente VIP con cupón ENVIO10, 5 kg urbano lunes	calculo el costo	se aplica 15 % y luego 10 %: 6.885 → **$6.900**
CA-3.2	un cliente frecuente un sábado, 5 kg urbano	calculo el costo	(9.000 + 2.000) × 0,95 = 10.450 → **$10.500** ($50 sube)
CA-3.3	un VIP con ENVIO10, 1 kg urbano lunes	calculo el costo	4.590 es menor que el mínimo → **$5.000**
CA-3.4	un cálculo que da 40.992	redondeo	el resultado es **$41.000** (múltiplo de 100 más cercano)

Matriz de trazabilidad

Requisito → historia → criterios → casos de prueba (caja negra TC, caja blanca CB, unitarias UT, cambio INS).

Requisito	Historia	Criterios	Casos de caja negra	Casos de caja blanca / unitarios	Defecto encontrado
R1 Peso válido	HU-01	CA-1.1, 1.2, 1.3	TC01, TC02, TC03, TC04, TC05	CB02, CB03, UT10	DEF-07
R2 Zona y día válidos	HU-01	CA-1.4, 1.5	TC13, TC14, TC15	CB08	DEF-04, DEF-05
R3 Tarifa base	HU-02	CA-2.1, 2.2, 2.3	TC06–TC12, TC35	CB01, CB04, CB05, UT06	DEF-01
R4 Recargo zona	HU-02	CA-2.4, 2.5	TC16, TC17, TC18, TC35	CB06, CB07, UT02–UT05	—
R5 Fin de semana	HU-02	CA-2.6	TC19, TC20, TC21, TC26–TC29	CB09, UT07	DEF-02
R6 Descuentos	HU-03	CA-3.1, 3.2	TC22–TC29, TC36	CB10, CB11, CB12, UT08	DEF-06 (redondeo)
R7 Cupón inexistente	HU-01 / HU-03	CA-1.6	TC30	CB13	DEF-03
R8 Mínimo y redondeo	HU-03	CA-3.2, 3.3, 3.4	TC10, TC12, TC22, TC23, TC25, TC28, TC29, TC31–TC34	CB14, UT09	DEF-06
CR-01 Zona insular	HU-04 (Parte 8)	CA-4.1–4.3	INS01–INS06	—	—

(La matriz se completó en las partes 3, 4, 5 y 8.)

Parte 2 · Plan de pruebas

Parte 2 · Plan de pruebas

1. Alcance

- **Objeto de prueba:** la función `calc(peso, zona, cliente, dia, cupon)` del módulo `envios.js` de EnvíaYa (versión heredada 1.0.0 y las versiones corregidas 1.0.1, 2.0.0 y 2.1.0).
- **Incluye:** las reglas R1–R8 de la especificación personalizada (código 0182410125), los valores límite de peso (0, 3, 10, 50), la combinación cliente × cupón × fin de semana, el mínimo de $5.000 y el redondeo, y la solicitud de cambio CR-01 (zona insular).
- **No incluye:** interfaz gráfica, base de datos, facturación, rendimiento y seguridad, que no forman parte del componente entregado.

2. Niveles de prueba aplicados a este caso

Nivel	Qué se prueba aquí	Cómo
Unidad	Cada función por separado: `tarifaBase`, `redondear`, `esFinDeSemana`, `aplicarDescuentos` y cada clase `Zona*` del patrón Estrategia.	Jest, `tests/estrategias.test.js` (UT01–UT10).
Integración	La colaboración entre las funciones extraídas y las estrategias de zona dentro de `calc` (el orden zona → fin de semana → cliente → cupón → mínimo → redondeo).	Casos de caja blanca CB01–CB15 y casos integrales TC33, TC34, que recorren todo el flujo.
Sistema	El componente completo frente a la especificación R1–R8, como caja negra, a través de su interfaz pública `calc`.	Matriz de 36 casos TC01–TC36 (`tests/cajaNegra.*.test.js`).
Aceptación	Los criterios Dado/Cuando/Entonces de HU-01, HU-02 y HU-03 (y HU-04 de la Parte 8), validados con el negocio.	Cada criterio está trazado a un caso automatizado (matriz de trazabilidad); se revisan con el "cliente" (docente) en la sustentación.

3. Estrategia

- **Caja negra:** particiones de equivalencia, análisis de valores límite y tabla de decisión. Los resultados esperados se **calculan a mano** a partir de la especificación, nunca ejecutando el código heredado.
- **Caja blanca:** grafo de flujo, complejidad ciclomática y un caso por camino independiente; medición de cobertura de sentencias y de ramas con Jest (`--coverage`, motor istanbul).
- **Regresión:** las mismas pruebas se ejecutan después de cada corrección (una rama `fix/DEF-xx` por defecto), después de la refactorización y después de la solicitud de cambio.
- **TDD para cambios:** en la Parte 8 y en la sustentación, primero se escribe la prueba que falla (rojo), luego el código (verde).

4. Ambiente

Elemento	Detalle
Sistema operativo	Windows 11 Pro
Lenguaje	JavaScript, Node.js v26 (traducción fiel de `envios.py`, defectos incluidos)
Framework	Jest 29.7.0 (pruebas y cobertura)
Control de versiones	Git 2.54 + GitHub (`calo159/parcialll`)
Datos	Valores fijos definidos en la matriz de casos; no se requiere base de datos

5. Roles

Rol	Responsable	Responsabilidad
Líder de calidad	Carlos David Sierra	Plan, diseño de casos, métricas e informe
Probador	Carlos David Sierra	Automatización y ejecución de pruebas, reporte de defectos
Desarrollador de mantenimiento	Carlos David Sierra	Corrección, refactorización y solicitud de cambio
Cliente / aceptación	Docente (EnvíaYa)	Valida criterios de aceptación y la sustentación

6. Riesgos y mitigación

Riesgo	Prob.	Impacto	Mitigación
Resultados esperados mal calculados a mano	Media	Alto	Cálculo paso a paso de casos clave; doble verificación de los límites
Errores de punto flotante (p. ej. 10449,999…) que cambian el redondeo	Alta	Medio	Normalizar a 6 decimales antes de redondear; casos límite con $50 exactos (TC28, UT09)
Ambigüedad de la especificación (cliente desconocido, día no entero)	Media	Medio	Supuestos documentados en la especificación
La refactorización introduce regresiones	Media	Alto	Ejecutar la suite completa después de cada commit; cobertura ≥ 95 % de ramas
Mala traducción de Python a JavaScript	Baja	Alto	Traducción línea a línea; mismos operadores (`<=`, `>`, truncamiento)
Falta de tiempo en la jornada	Media	Alto	Commits pequeños e incrementales; priorizar defectos de severidad alta

7. Criterios de entrada

- La especificación R1–R8 está escrita con los valores del código 0182410125.
- `CONFIG` tiene los valores personales y el código heredado tiene la etiqueta `v1.0.0`.
- El ambiente (Node + Jest) está instalado y `npm test` se ejecuta.
- La matriz de casos tiene los resultados esperados calculados a mano.

8. Criterios de salida

- El 100 % de los casos diseñados están automatizados y ejecutados.
- Todos los defectos de severidad alta y media están corregidos y verificados (0 abiertos).
- La regresión final está **100 % en verde**.
- Cobertura de sentencias = 100 % y de ramas ≥ 95 % sobre `envios.js`.

- El CHANGELOG y las etiquetas de versión están publicados.

Parte 3 · Diseño de casos de caja negra

Parte 3 · Diseño de casos de caja negra

3.1 Particiones de equivalencia

Entrada	Clases válidas	Clases inválidas
peso	V1: 0 < peso < 3 (tramo 1) · V2: 3 ≤ peso ≤ 10 (tramo 2) · V3: 10 < peso ≤ 50 (tramo 3)	I1: peso ≤ 0 · I2: peso > 50 · I3: no numérico ("5")
zona	V4: urbana · V5: rural · V6: nacional	I4: cualquier otro texto ("marte", "Rural", "")
cliente	V7: normal · V8: frecuente · V9: vip	(no especificado: se trata como normal, ver supuesto)
dia	V10: 1–5 (entre semana) · V11: 6–7 (fin de semana)	I5: dia < 1 · I6: dia > 7
cupon	V12: sin cupón (null) · V13: "ENVIO10"	I7: cupón inexistente ("GRATIS", "XYZ") → error (F impar)

Casos que cubren cada clase: V1 TC06 · V2 TC07, TC09 · V3 TC11 · V4 TC21 · V5 TC16 · V6 TC17 · V7 TC24 · V8 TC22 · V9 TC23 · V10 TC21 · V11 TC19, TC20 · V12 TC22 · V13 TC24 · I1 TC01, TC02 · I2 TC03 · I3 UT10 · I4 TC13 · I5 TC14 · I6 TC15 · I7 TC30.

3.2 Valores límite de peso

L1 = 3 kg **no está incluido** en el tramo 1 (D = 1 impar → peso < 3). L2 = 10 kg **sí** está incluido en el tramo 2. PMAX = 50 kg.

Límite	Valor probado	Clase	Esperado	Caso
0	0	inválido	error "peso inválido"	TC01
0	0,01	tramo 1	$6.000	TC05
L1 = 3	2,99	tramo 1	$6.000	TC06
L1 = 3	**3**	**tramo 2** (límite excluido del tramo 1)	**$9.000**	TC07, TC35
L1 = 3	3,01	tramo 2	$9.000	TC08
L2 = 10	10	tramo 2	$9.000	TC09
L2 = 10	10,01	tramo 3	9.008 → $9.000	TC10
PMAX = 50	49,99	tramo 3	40.992 → $41.000	TC12
PMAX = 50	50	tramo 3	$41.000	TC04
PMAX = 50	50,01	inválido	error	TC03

Valores límite de día: 0 (error, TC14), 5 (sin recargo, TC21), 6 (recargo, TC19), 7 (recargo, TC20), 8 (error, TC15).

3.3 Tabla de decisión: cliente × cupón × fin de semana

Reglas: **E = 2 par** → el recargo de $2.000 se suma **antes** de los descuentos. **B = 1 impar** → se aplican **ambos** descuentos: primero el del cliente y luego el cupón sobre el resultado. Base de todas las columnas: 5 kg, urbana

→ $9.000.

Condiciones / Acciones	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12
Cliente	normal	normal	normal	normal	frec.	frec.	frec.	frec.	vip	vip	vip	vip
Cupón ENVIO10	No	No	Sí	Sí	No	No	Sí	Sí	No	No	Sí	Sí
Fin de semana	No	Sí	No	Sí	No	Sí	No	Sí	No	Sí	No	Sí
Sumar $2.000 antes	–	X	–	X	–	X	–	X	–	X	–	X
Descuento cliente	–	–	–	–	5 %	5 %	5 %	5 %	15 %	15 %	15 %	15 %
Luego cupón 10 %	–	–	X	X	–	–	X	X	–	–	X	X
Cálculo	9.000	11.000	8.100	9.900	8.550	10.450	7.695	9.405	7.650	9.350	6.885	8.415
Resultado	**9.000**	**11.000**	**8.100**	**9.900**	**8.600**	**10.500**	**7.700**	**9.400**	**7.700**	**9.400**	**6.900**	**8.400**
Caso	CB01	TC19	TC24	(D4)	TC22	TC28	TC36	TC26	TC23	TC29	TC25	TC27

Con 3 condiciones binarias (cliente con 3 valores) hay 3 × 2 × 2 = 12 reglas; todas están cubiertas por al menos un caso automatizado, excepto D4, que se verifica de forma implícita con TC24 + TC19 (la regla D4 es la composición de ambas).

Parte 3 · Matriz de casos de prueba de caja negra (36 casos)

Técnicas: **PE** = partición de equivalencia · **VL** = valor límite · **TD** = tabla de decisión · **INT** = caso integral. Entradas: (peso, zona, cliente, día, cupón). Los resultados esperados se calcularon a mano desde la especificación.

ID	Req.	Entradas	Cálculo a mano	Esperado	Técnica
TC01	R1	(0, urbana, normal, 1)	0 no cumple 0 < peso	error "peso inválido"	VL
TC02	R1	(-1, urbana, normal, 1)	negativo	error	PE (I1)
TC03	R1	(50.01, urbana, normal, 1)	50,01 > PMAX	error	VL
TC04	R1, R3	(50, urbana, normal, 1)	9.000 + 40 × 800 = 41.000	$41.000	VL
TC05	R1, R3	(0.01, urbana, normal, 1)	tramo 1	$6.000	VL
TC06	R3	(2.99, urbana, normal, 1)	2,99 < 3 → tramo 1	$6.000	VL
TC07	R3	(3, urbana, normal, 1)	3 no es < 3 → tramo 2	$9.000	VL
TC08	R3	(3.01, urbana, normal, 1)	tramo 2	$9.000	VL
TC09	R3	(10, urbana, normal, 1)	10 ≤ 10 → tramo 2	$9.000	VL
TC10	R3, R8	(10.01, urbana, normal, 1)	9.000 + 0,01 × 800 = 9.008 → 9.000	$9.000	VL
TC11	R3	(10.5, urbana, normal, 1)	9.000 + 0,5 × 800 = 9.400	$9.400	PE (V3)
TC12	R3, R8	(49.99, urbana, normal, 1)	9.000 + 39,99 × 800 = 40.992 → 41.000	$41.000	VL
TC13	R2	(5, marte, normal, 1)	zona no válida	error	PE (I4)
TC14	R2	(5, urbana, normal, 0)	día < 1	error	VL
TC15	R2	(5, urbana, normal, 8)	día > 7	error	VL
TC16	R4	(5, rural, normal, 1)	9.000 × 1,20	$10.800	PE (V5)
TC17	R4	(5, nacional, normal, 1)	9.000 × 1,50	$13.500	PE (V6)

ID	Req.	Entradas	Cálculo a mano	Esperado	Técnica
TC18	R4	(1, nacional, normal, 1)	6.000 × 1,50	$9.000	PE
TC19	R5	(5, urbana, normal, 6)	9.000 + 2.000	$11.000	VL
TC20	R5	(5, urbana, normal, 7)	9.000 + 2.000	$11.000	VL
TC21	R5	(5, urbana, normal, 5)	sin recargo	$9.000	VL
TC22	R6, R8	(5, urbana, frecuente, 1)	9.000 × 0,95 = 8.550 → 8.600	$8.600	TD (D5)
TC23	R6, R8	(5, urbana, vip, 1)	9.000 × 0,85 = 7.650 → 7.700	$7.700	TD (D9)
TC24	R6	(5, urbana, normal, 1, ENVIO10)	9.000 × 0,90	$8.100	TD (D3)
TC25	R6, R8	(5, urbana, vip, 1, ENVIO10)	9.000 × 0,85 × 0,90 = 6.885 → 6.900	$6.900	TD (D11)
TC26	R5, R6	(5, urbana, frecuente, 7, ENVIO10)	11.000 × 0,95 × 0,90 = 9.405 → 9.400	$9.400	TD (D8)
TC27	R5, R6	(5, urbana, vip, 6, ENVIO10)	11.000 × 0,85 × 0,90 = 8.415 → 8.400	$8.400	TD (D12)
TC28	R5, R8	(5, urbana, frecuente, 6)	11.000 × 0,95 = 10.450 → 10.500	$10.500	TD (D6)
TC29	R5, R8	(5, urbana, vip, 7)	11.000 × 0,85 = 9.350 → 9.400	$9.400	TD (D10)
TC30	R7	(5, urbana, normal, 1, GRATIS)	cupón no existe, F impar	error	PE (I7)
TC31	R8	(1, urbana, vip, 1, ENVIO10)	6.000 × 0,85 × 0,90 = 4.590 < 5.000	$5.000	VL
TC32	R8	(2, urbana, frecuente, 1, ENVIO10)	6.000 × 0,95 × 0,90 = 5.130 → 5.100	$5.100	VL
TC33	R3–R8	(4, rural, vip, 6, ENVIO10)	ver cálculo 1	$9.800	INT
TC34	R3–R8	(12.5, nacional, frecuente, 7, ENVIO10)	ver cálculo 2	$15.800	INT
TC35	R3, R4	(3, rural, normal, 1)	9.000 × 1,20	$10.800	VL
TC36	R6, R8	(5, urbana, frecuente, 1, ENVIO10)	9.000 × 0,95 × 0,90 = 7.695 → 7.700	$7.700	TD (D7)

Cálculos paso a paso (mínimo 5)

Cálculo 1 · TC33 — (4 kg, rural, vip, sábado, ENVIO10)

1. Tramo: D = 1 impar, el tramo 1 es peso < 3. 4 no es menor que 3 y 4 ≤ 10 → tramo 2 → **$9.000**
2. Zona rural +20 % → 9.000 × 1,20 = **$10.800**
3. Sábado y E = 2 par: el recargo va antes de los descuentos → 10.800 + 2.000 = **$12.800**
4. B = 1 impar: ambos descuentos. VIP 15 % → 12.800 × 0,85 = $10.880. Cupón 10 % → 10.880 × 0,90 = **$9.792**
5. Es mayor que $5.000. Se redondea a $100 → **resultado esperado: $9.800**

Cálculo 2 · TC34 — (12,5 kg, nacional, frecuente, domingo, ENVIO10)

1. 12,5 > 10 → tramo 3: 9.000 + (12,5 − 10) × 800 = 9.000 + 2.000 = **$11.000**
2. Zona nacional +50 % → 11.000 × 1,50 = **$16.500**
3. Domingo, E par → 16.500 + 2.000 = **$18.500**
4. Frecuente 5 % → 18.500 × 0,95 = $17.575. Cupón 10 % → 17.575 × 0,90 = **$15.817,50**
5. $5.000. El múltiplo de $100 más cercano a 15.817,50 es 15.800 (faltan 17,50 para bajar y 82,50 para subir) → **$15.800**

Cálculo 3 · TC28 — (5 kg, urbana, frecuente, sábado, sin cupón)

1. 3 ≤ 5 ≤ 10 → tramo 2 → **$9.000**
2. Urbana, sin recargo → **$9.000**
3. Sábado, E par → 9.000 + 2.000 = **$11.000**
4. Frecuente 5 % → 11.000 × 0,95 = **$10.450**; sin cupón
5. $5.000. 10.450 está justo en la mitad: los $50 redondean hacia arriba → **$10.500**

Cálculo 4 · TC25 — (5 kg, urbana, vip, lunes, ENVIO10)

1. Tramo 2 → **$9.000**
2. Urbana → **$9.000**
3. Lunes, sin recargo → **$9.000**
4. B impar: VIP 15 % → 9.000 × 0,85 = $7.650; cupón 10 % → 7.650 × 0,90 = **$6.885**
5. $5.000. 6.885 → el múltiplo de 100 más cercano es **$6.900**

Cálculo 5 · TC31 — (1 kg, urbana, vip, lunes, ENVIO10)

1. 1 < 3 → tramo 1 → **$6.000**
2. Urbana → **$6.000**; lunes, sin recargo
3. VIP 15 % → 6.000 × 0,85 = $5.100; cupón 10 % → 5.100 × 0,90 = **$4.590**
4. 4.590 < 5.000 → se sube al mínimo **$5.000**
5. Redondeo: 5.000 ya es múltiplo de 100 → **$5.000**

Cálculo 6 · TC12 — (49,99 kg, urbana, normal, lunes)

1. 49,99 > 10 → 9.000 + 39,99 × 800 = 9.000 + 31.992 = **$40.992**
2. Urbana, lunes, normal, sin cupón → **$40.992**
3. Redondeo al múltiplo de 100 más cercano: 40.992 está a 8 de 41.000 → **$41.000**

Parte 4 · Caja blanca

Código heredado analizado (envios.js v1.0.0, con números de línea en el editor)

```
// envios.js — código heredado v1.0.0 (traducción fiel de envios.py)
const CONFIG = {
  L1: 3, // <- pon tus valores
  L2: 10,
  RECARGO_RURAL: 0.2, // como fracción, ej. 0.15
  RECARGO_NACIONAL: 0.5,
  DESC_VIP: 0.15,
  RECARGO_FINDE: 2000, // en pesos
};

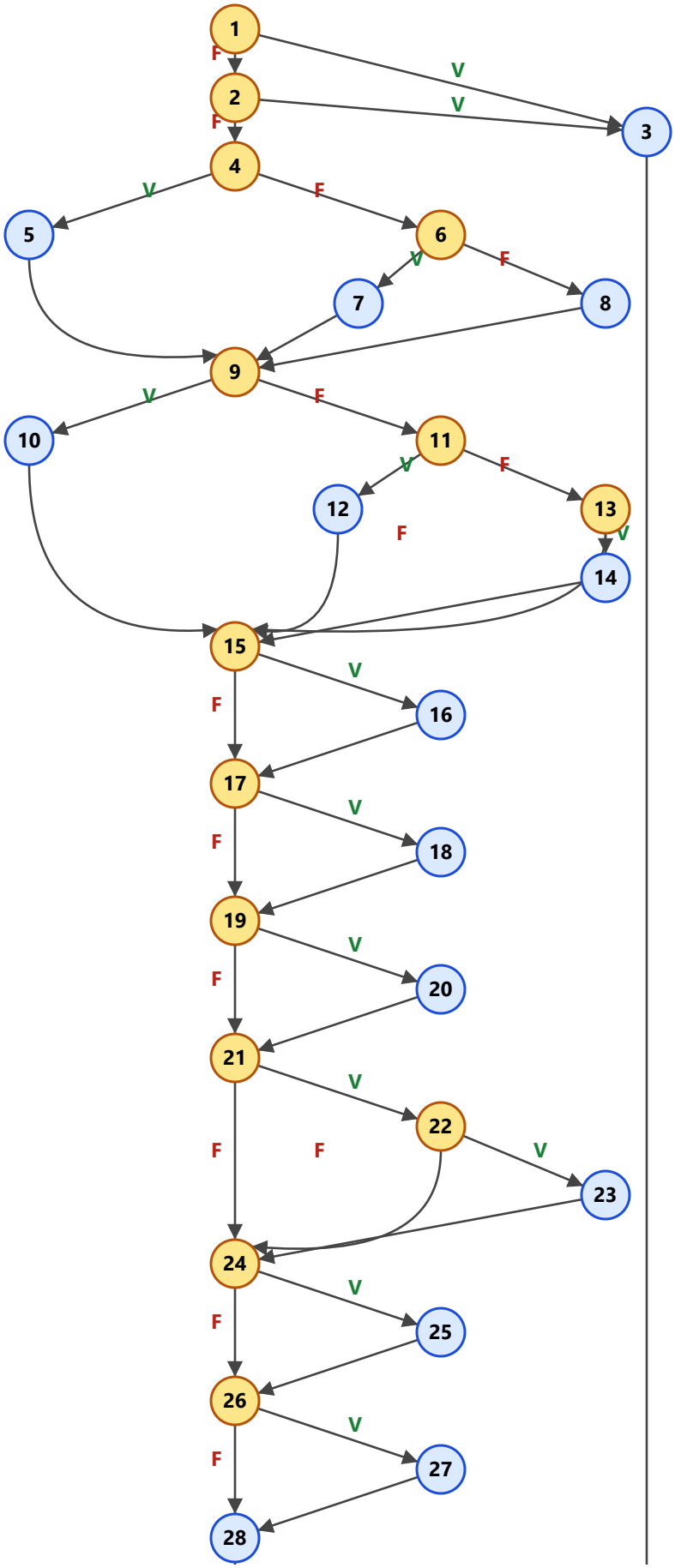
function calc(p, z, c, d, cup = null, flag = false) {
  let t;
  if (p <= 0 || p > 50) {
    throw new Error("peso invalido");
  }
  if (p <= CONFIG.L1) {
    t = 6000;
  } else if (p <= CONFIG.L2) {
    t = 9000;
  } else {
    t = 9000 + (p - CONFIG.L2) * 800;
  }
  if (z == "urbana") {
    t = t;
  } else if (z == "rural") {
    t = t + t * CONFIG.RECARGO_RURAL;
  } else if (z == "nacional") {
    t = t + t * CONFIG.RECARGO_NACIONAL;
  }
  // fin de semana
  if (d > 6) {
    t = t + CONFIG.RECARGO_FINDE;
  }
  if (c == "frecuente") {
    t = t - t * 0.05;
  }
  if (c == "vip") {
    t = t - t * CONFIG.DESC_VIP;
  }
  if (cup != null) {
    if (cup == "ENVIO10") {
      t = t - t * 0.1;
    }
  }
  if (t < 5000) {
    t = 5000;
  }
  if (flag) {
    console.log("costo:", t);
  }
  return Math.trunc(t / 100) * 100;
}

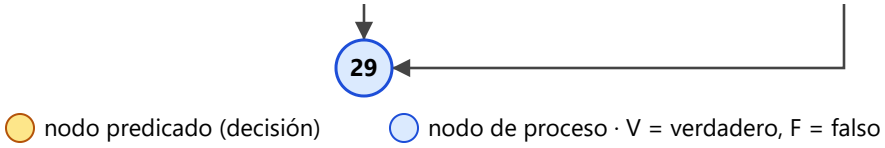
module.exports = { calc, CONFIG };
```

Parte 4 · Caja blanca

4.1 Grafo de flujo de `calc` (código heredado v1.0.0)

Grafo de flujo de calc (v1.0.0) · N = 29, E = 42, P = 14, V(G) = 15





La condición compuesta `p <= 0 || p > 50` se separa en dos nodos predicado (1 y 2), como se hace en un grafo de flujo.

Nodo	Línea(s) de `envios.js` v1.0.0	Contenido	Tipo
1	13	`p <= 0`	predicado
2	13	`p > 50`	predicado
3	14	`throw new Error("peso invalido")`	proceso
4	16	`p <= CONFIG.L1`	predicado
5	17	`t = 6000`	proceso
6	18	`p <= CONFIG.L2`	predicado
7	19	`t = 9000`	proceso
8	21	`t = 9000 + (p - L2) * 800`	proceso
9	23	`z == "urbana"`	predicado
10	24	`t = t`	proceso
11	25	`z == "rural"`	predicado
12	26	recargo rural	proceso
13	27	`z == "nacional"`	predicado
14	28	recargo nacional	proceso
15	31	`d > 6`	predicado
16	32	`t + RECARGO_FINDE`	proceso
17	34	`c == "frecuente"`	predicado
18	35	descuento 5 %	proceso
19	37	`c == "vip"`	predicado
20	38	descuento VIP	proceso
21	40	`cup != null`	predicado
22	41	`cup == "ENVIO10"`	predicado
23	42	descuento 10 %	proceso
24	45	`t < 5000`	predicado
25	46	`t = 5000`	proceso
26	48	`flag`	predicado
27	49	`console.log`	proceso
28	51	`return Math.trunc(t / 100) * 100`	proceso
29	—	fin (salida única)	fin

4.2 Complejidad ciclomática de tres formas

1. **Con aristas y nodos:** E = 42 aristas, N = 29 nodos → V(G) = E − N + 2 = 42 − 29 + 2 = **15**
2. **Con nodos predicado:** P = 14 (nodos 1, 2, 4, 6, 9, 11, 13, 15, 17, 19, 21, 22, 24, 26) → V(G) = P + 1 = **15**
3. **Con regiones:** el grafo es plano y tiene 14 regiones cerradas más la región exterior → R = **15** (cada decisión binaria cierra una región: 2 por la validación del peso, 2 por los tramos, 3 por las zonas, 1 por el fin de semana, 2 por los clientes, 2 por el cupón, 1 por el mínimo y 1 por el flag).

V(G) = 15 → se necesitan como máximo 15 caminos independientes, y como mínimo 15 casos para cubrirlos. (Si la condición compuesta del peso se contara como un solo predicado, V(G) = 14; se usa 15 porque el grafo separa las condiciones.)

4.3 Caminos independientes y casos de prueba

Camino base: 1-2-4-6-7-9-10-15-17-19-21-24-26-28-29. Cada camino siguiente agrega al menos una arista nueva.

Camino	Secuencia de nodos (arista nueva en negrita)	Caso	Entradas	Esperado (especificación)
C1	1-2-4-6-7-9-10-15-17-19-21-24-26-28-29	CB01	(5, urbana, normal, 1)	$9.000
C2	**1-3**-29	CB02	(0, urbana, normal, 1)	error
C3	1-**2-3**-29	CB03	(51, urbana, normal, 1)	error
C4	1-2-**4-5-9**-10-15-...-29	CB04	(2, urbana, normal, 1)	$6.000
C5	1-2-4-**6-8-9**-10-15-...-29	CB05	(12, urbana, normal, 1)	$10.600
C6	...-9-**11-12-15**-17-...-29	CB06	(5, rural, normal, 1)	$10.800
C7	...-9-11-**13-14-15**-...-29	CB07	(5, nacional, normal, 1)	$13.500
C8	...-9-11-13-**15** (ninguna zona)	CB08	(5, marte, normal, 1)	error (R2)
C9	...-**15-16-17**-...-29	CB09	(5, urbana, normal, 7)	$11.000
C10	...-**17-18-19**-...-29	CB10	(5, urbana, frecuente, 1)	$8.600
C11	...-**19-20-21**-...-29	CB11	(5, urbana, vip, 1)	$7.700
C12	...-**21-22-23-24**-...-29	CB12	(5, urbana, normal, 1, ENVIO10)	$8.100
C13	...-21-**22-24**-...-29	CB13	(5, urbana, normal, 1, XYZ)	error (R7)
C14	...-**24-25-26**-...-29	CB14	(1, urbana, vip, 1, ENVIO10)	$5.000
C15	...-**26-27-28**-29	CB15	(5, urbana, normal, 1, null, true)	imprime "costo: 9000" y retorna $9.000

Nota: C8 y C13 existen en el código heredado (la zona o el cupón no coinciden y el flujo continúa), pero la especificación pide un error. Por eso CB08 y CB13 fallan contra la v1.0.0 y revelan DEF-04 y DEF-03.

4.4 Cobertura de sentencias y de ramas

- **Cobertura de sentencias (100 %):** basta con un subconjunto que ejecute todas las sentencias: CB02 (throw), CB04 (t = 6000), CB01 (t = 9000), CB05 (tramo 3), CB06 (rural), CB07 (nacional), CB09 (fin de

semana), CB10 (frecuente), CB11 (VIP), CB12 (cupón), CB14 (t = 5000) y CB15 (console.log). Esos **12 casos** cubren todas las sentencias.

- **Cobertura de ramas (100 %):** además de los anteriores se necesita que cada decisión tome su salida falsa y verdadera: CB03 (rama verdadera de `p > 50`), CB08 (salida falsa de `z == "nacional"`) y CB13 (salida falsa de `cup == "ENVIO10"`). Con los **15 casos CB01–CB15** se recorren las 28 ramas (14 decisiones × 2).
- **Medición real con Jest** sobre `envios.js` v1.0.0 (toda la suite, 51 casos): **100 % de sentencias, 100 % de ramas, 100 % de funciones y 100 % de líneas**: 29/29 sentencias y 30/30 ramas; istanbul cuenta 30 ramas porque, además de las 28 salidas de las decisiones, incluye los dos lados del operador `||` (ver `evidencias/01_heredado.png`). Que la cobertura sea del 100 % y aun así fallen 21 pruebas muestra que la cobertura no garantiza la corrección: los defectos son de lógica contra la especificación.

Parte 5 · Automatización, ejecución y defectos

Parte 5 · Automatización, ejecución y defectos

5.1 Automatización

- Framework: **Jest 29.7.0** sobre Node.js (el enunciado permite JavaScript siempre que la traducción conserve la lógica y los defectos).
- Archivos de prueba:
  - `tests/cajaNegra.validaciones.test.js` : TC01–TC15 y TC35 (R1, R2, R3)
  - `tests/cajaNegra.reglas.test.js` : TC16–TC34 y TC36 (R4–R8)
  - `tests/cajaBlanca.test.js` : CB01–CB15 (un caso por camino independiente)
- Ejecución: `npm test` · cobertura: `npm run test:cov` (Jest `--coverage`, motor istanbul: sentencias, ramas, funciones y líneas).

5.2 Ejecución contra el código heredado (v1.0.0)

Resultado: **51 pruebas · 30 pasan · 21 fallan** (captura: `evidencias/01_heredado.png`).

Caso que falla	Esperado	Obtenido	Defecto
TC01	error "peso inválido"	error "peso invalido"	DEF-07
TC07	9.000	6.000	DEF-01
TC35	10.800	7.200	DEF-01
TC12	41.000	40.900	DEF-06
TC13, CB08	error	9.000	DEF-04
TC14	error	9.000	DEF-05
TC15	error	11.000	DEF-05
TC19	11.000	9.000	DEF-02
TC27	8.400	6.800	DEF-02
TC28	10.500	8.500	DEF-02 + DEF-06
TC33	9.800	8.200	DEF-02
TC22, CB10	8.600	8.500	DEF-06
TC23, CB11	7.700	7.600	DEF-06
TC25	6.900	6.800	DEF-06
TC29	9.400	9.300	DEF-06
TC36	7.700	7.600	DEF-06
TC30, CB13	error	9.000	DEF-03

5.3 Reporte de defectos

Escala de **severidad** (impacto técnico/negocio): Alta = precio incorrecto en un caso frecuente o aceptación silenciosa de datos inválidos; Media = precio o validación incorrectos con impacto acotado; Baja = cosmético. **Prioridad** (urgencia de corrección): Alta = corregir antes de la próxima liberación; Media = en esta versión; Baja = cuando sea posible.

DEF-01 · El tramo 1 incluye el límite L1 = 3 kg

- **Requisito:** R3 (D = 1 impar → tramo 1 es peso < 3)
- **Pasos:** 1) `calc(3, "urbana", "normal", 1)` 2) observar el resultado
- **Esperado:** $9.000 (3 kg pertenece al tramo 2) · **Obtenido:** $6.000
- **Línea:** 16 `if (p <= CONFIG.L1)`
- **Severidad: Alta.** Se cobra $3.000 menos (33 %) en todos los paquetes de exactamente 3 kg, un peso redondo muy común.
- **Prioridad: Alta.** Pérdida directa de ingresos en cada envío afectado; la corrección es de un carácter.

DEF-02 · El sábado no cobra el recargo de fin de semana

- **Requisito:** R5 (sábado 6 y domingo 7 → +$2.000)
- **Pasos:** `calc(5, "urbana", "normal", 6)`
- **Esperado:** $11.000 · **Obtenido:** $9.000
- **Línea:** 31 `if (d > 6)` solo considera el domingo
- **Severidad: Alta.** Todos los envíos de sábado salen sin recargo; afecta la mitad de los envíos de fin de semana.
- **Prioridad: Alta.** Pérdida de ingresos recurrente cada semana.

DEF-03 · Un cupón inexistente se ignora en vez de lanzar error

- **Requisito:** R7 (F = 5 impar → cupón inexistente es error)
- **Pasos:** `calc(5, "urbana", "normal", 1, "GRATIS")`
- **Esperado:** error de cupón inválido · **Obtenido:** $9.000, sin error
- **Línea:** 40–44 (no hay rama `else` para un cupón desconocido)
- **Severidad: Media.** El precio no se altera, pero el cliente cree que su cupón se aplicó; se oculta un error de digitación.
- **Prioridad: Media.** Genera reclamos de clientes, pero sin pérdida económica directa.

DEF-04 · Una zona inválida no lanza error

- **Requisito:** R2
- **Pasos:** `calc(5, "marte", "normal", 1)` (o `"Rural"` con mayúscula)
- **Esperado:** error de zona inválida · **Obtenido:** $9.000 (se cobra como urbana)
- **Línea:** 23–29 (la cadena `if/else if` no tiene `else`)
- **Severidad: Alta.** Un error de digitación en la zona hace que un envío nacional se cobre como urbano (−33 % del precio) sin aviso.
- **Prioridad: Alta.** Es silencioso y fácil de provocar.

DEF-05 · Un día fuera de 1..7 no lanza error

- **Requisito:** R2

- **Pasos:** `calc(5, "urbana", "normal", 8)` y `calc(5, "urbana", "normal", 0)`
- **Esperado:** error de día inválido · **Obtenido:** $11.000 (el 8 se trata como fin de semana) y $9.000
- **Línea:** 11–15 (no se valida `d`)
- **Severidad: Media.** Se aceptan datos imposibles; el precio puede salir más alto o más bajo.
- **Prioridad: Media.** Normalmente el día lo genera el sistema, así que la entrada inválida es menos probable.

DEF-06 · El redondeo trunca en lugar de ir al múltiplo de $100 más cercano

- **Requisito:** R8 (los $50 redondean hacia arriba)
- **Pasos:** `calc(5, "urbana", "frecuente", 1)` (8.550) y `calc(49.99, "urbana", "normal", 1)` (40.992)
- **Esperado:** $8.600 y $41.000 · **Obtenido:** $8.500 y $40.900
- **Línea:** 51 `return Math.trunc(t / 100) * 100` (en Python, `int(t / 100) * 100`)
- **Severidad: Media.** Hasta $99 de diferencia por envío.
- **Prioridad: Alta.** Afecta a cerca de la mitad de los envíos con descuento y es el defecto que más pruebas hace fallar (9).

DEF-07 · El mensaje de error del peso no coincide con la especificación

- **Requisito:** R1 (error **"peso inválido"**)
- **Pasos:** `calc(0, "urbana", "normal", 1)`
- **Esperado:** mensaje "peso inválido" · **Obtenido:** "peso invalido"
- **Línea:** 14
- **Severidad: Baja.** Cosmético, pero rompe a cualquier cliente que compare el mensaje.
- **Prioridad: Baja.** Se corrige junto con las demás por su bajo costo.

Observaciones (no son defectos con mi especificación)

- `p > 50`: el máximo del código heredado (50) **coincide** con mi PMAX = 50, así que no es defecto.
- El recargo de fin de semana se suma **antes** de los descuentos (E par) y los descuentos se aplican **en secuencia** (B impar): el código heredado ya hace ambas cosas, así que no son defectos para mí.
- Un cliente desconocido no recibe descuento: la especificación no lo define (se documenta como supuesto).

5.4 Métricas

Métrica	Valor
Casos ejecutados	51
Casos que pasan	30 → **58,8 %**
Casos que fallan	21 → 41,2 %
Defectos por severidad	**Alta: 3** (DEF-01, 02, 04) · **Media: 3** (DEF-03, 05, 06) · **Baja: 1** (DEF-07) · Total: 7
Densidad de defectos	7 defectos / 54 líneas × 100 = **12,96 defectos por cada 100 líneas**
Cobertura de sentencias (heredado)	**100 %** (29/29)
Cobertura de ramas (heredado)	**100 %** (30/30)
Cobertura de funciones / líneas	100 % / 100 %

Evidencia: pruebas fallando contra el código heredado (v1.0.0)

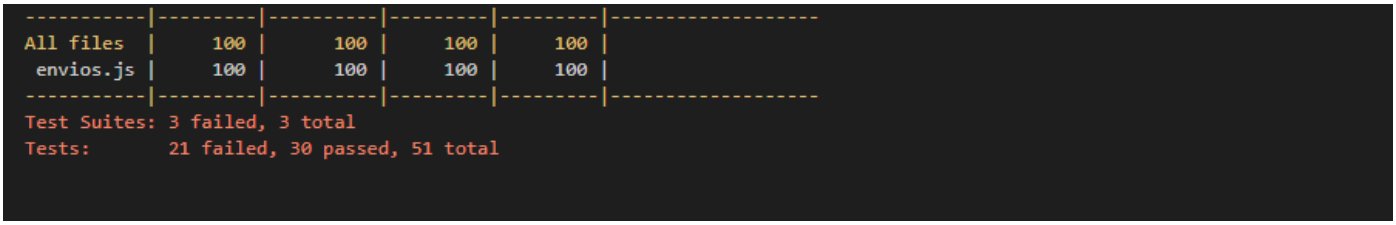
```
PS C:\parcialIII> npm run test:cov # contra código heredado v1.0.0

FAIL tests/cajaBlanca.test.js
  Caminos independientes de calc
    √ CB01 camino base: 5 kg urbana normal lunes -> 9000 (7 ms)
    √ CB02 peso <= 0 verdadero -> error (10 ms)
    √ CB03 peso > 50 verdadero -> error (1 ms)
    √ CB04 tramo 1: 2 kg -> 6000
    √ CB05 tramo 3: 12 kg -> 9000 + 2*800 = 10600 (1 ms)
    √ CB06 zona rural -> 10800
    √ CB07 zona nacional -> 13500
    x CB08 ninguna zona coincide ('marte') -> error (1 ms)
    √ CB09 fin de semana (domingo) -> 11000 (1 ms)
    x CB10 cliente frecuente -> 8550 -> 8600 (2 ms)
    x CB11 cliente vip -> 7650 -> 7700 (1 ms)
    √ CB12 cupón ENVIO10 -> 8100 (1 ms)
    x CB13 cupón distinto de ENVIO10 ('XYZ') -> error
    √ CB14 costo < 5000: 1 kg vip ENVIO10 -> 5000 (1 ms)
    √ CB15 flag verdadero imprime el costo y retorna 9000 (1 ms)

FAIL tests/cajaNegra.reglas.test.js
  R4 - Recargo por zona
    √ TC16 [R4][PE] rural, 5 kg -> 9000 * 1.20 = 10800 (8 ms)
    √ TC17 [R4][PE] nacional, 5 kg -> 9000 * 1.50 = 13500 (1 ms)
    √ TC18 [R4][PE] nacional, 1 kg -> 6000 * 1.50 = 9000
  R5 - Recargo de fin de semana
    x TC19 [R5][VL] sábado (6) -> 9000 + 2000 = 11000 (2 ms)
    √ TC20 [R5][VL] domingo (7) -> 11000
    √ TC21 [R5][VL] viernes (5) -> sin recargo = 9000
  R6 - Tabla de decisión cliente x cupón x fin de semana
    x TC22 [R6][TD] frecuente, entre semana -> 8550 redondeado = 8600 (1 ms)
    x TC23 [R6][TD] vip, entre semana -> 7650 redondeado = 7700
    √ TC24 [R6][TD] normal + ENVIO10 -> 8100
    x TC25 [R6][TD] vip + ENVIO10 -> 9000*0.85*0.90 = 6885 -> 6900
    x TC36 [R6][TD] frecuente + ENVIO10 -> 9000*0.95*0.90 = 7695 -> 7700
    √ TC26 [R6][TD] frecuente + ENVIO10 + domingo -> 11000*0.95*0.90 = 9405 -> 9400
    x TC27 [R6][TD] vip + ENVIO10 + sábado -> 11000*0.85*0.90 = 8415 -> 8400 (1 ms)
    x TC28 [R6][TD] frecuente + sábado -> 11000*0.95 = 10450 -> 10500
    x TC29 [R6][TD] vip + domingo -> 11000*0.85 = 9350 -> 9400 (1 ms)
  R7 - Cupón inexistente
    x TC30 [R7][PE] cupón 'GRATIS' lanza error (F impar) (1 ms)
  R8 - Costo mínimo y redondeo
    √ TC31 [R8][VL] 1 kg vip + ENVIO10 -> 4590 sube al mínimo 5000
    √ TC32 [R8][VL] 2 kg frecuente + ENVIO10 -> 5130 -> 5100
  Casos integrales (todas las reglas)
    x TC33 [R3-R8] 4 kg rural vip sábado ENVIO10 -> 9792 -> 9800
    √ TC34 [R3-R8] 12.5 kg nacional frecuente domingo ENVIO10 -> 15817.5 -> 15800

FAIL tests/cajaNegra.validaciones.test.js
  R1 - Validación del peso
    x TC01 [R1][VL] peso = 0 lanza 'peso inválido' (29 ms)
    √ TC02 [R1][PE] peso negativo (-1) lanza error (1 ms)
    √ TC03 [R1][VL] peso = 50.01 (PMAX + 0.01) lanza error
    √ TC04 [R1][VL] peso = 50 (PMAX) es válido -> 41000 (1 ms)
    √ TC05 [R1][VL] peso = 0.01 es válido -> 6000 (1 ms)
  R3 - Valores límite de los tramos
    √ TC06 [R3][VL] peso = 2.99 (L1 - 0.01) -> tramo 1 = 6000
    x TC07 [R3][VL] peso = 3 (L1, excluido) -> tramo 2 = 9000 (1 ms)
    √ TC08 [R3][VL] peso = 3.01 (L1 + 0.01) -> 9000 (1 ms)
    √ TC09 [R3][VL] peso = 10 (L2, incluido) -> 9000
    √ TC10 [R3][VL] peso = 10.01 (L2 + 0.01) -> 9008 redondeado = 9000
    √ TC11 [R3][PE] peso = 10.5 -> 9000 + 0.5*800 = 9400 (1 ms)
    x TC12 [R3][VL] peso = 49.99 (PMAX - 0.01) -> 40992 redondeado = 41000
    x TC35 [R3][VL] peso = 3 rural -> 9000 * 1.20 = 10800 (1 ms)
  R2 - Validación de zona y día
    x TC13 [R2][PE] zona inexistente 'marte' lanza error
    x TC14 [R2][VL] día = 0 lanza error
    x TC15 [R2][VL] día = 8 lanza error

-----------|---------|----------|---------|---------|-------------------
File       | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
```



Captura: npm run test:cov # contra código heredado v1.0.0 — archivo completo en `evidencias/01_heredado.txt` y `.png`

Parte 6 · Mantenimiento y refactorización

Parte 6 · Mantenimiento y refactorización

6.1 Malos olores (code smells) del código heredado

Las líneas se refieren a `envios.js` v1.0.0 (traducción línea a línea de `envios.py`).

#	Mal olor (nombre técnico)	Línea(s)	Evidencia	Deuda técnica que genera
1	**Números mágicos** (*Magic Numbers*)	13 (`0`, `50`), 17 (`6000`), 19 y 21 (`9000`, `800`), 31 (`6`), 35 (`0.05`), 42 (`0.1`), 45–46 (`5000`), 51 (`100`)	Valores del negocio escritos como literales	Cambiar una tarifa obliga a buscar el número en todo el código; un cambio olvidado deja precios inconsistentes.
2	**Método largo / función que hace demasiado** (*Long Method*, violación de responsabilidad única)	11–52	`calc` valida, calcula la tarifa, aplica zona, fin de semana, descuentos, mínimo, redondeo e imprime	No se puede probar cada regla por separado; V(G) = 15 hace difícil entenderla y modificarla sin romper otra regla.
3	**Argumento bandera** (*Flag Argument*)	11 (`flag = false`), 48–50	Un booleano cambia el comportamiento (imprimir o no)	Mezcla cálculo con entrada/salida, agrega un camino más que probar y ensucia la interfaz pública.
4	**Nombres misteriosos** (*Mysterious Name*)	11 (`p, z, c, d, cup`), 12 (`t`)	Una letra por parámetro	Hay que leer todo el cuerpo para saber qué es cada variable; sube el costo de incorporar a un nuevo desarrollador.
5	**Código muerto / asignación inútil** (*Dead Code*, *self-assignment*)	24 `t = t`	La rama urbana no hace nada	Confunde al lector (¿falta algo?) y agrega una rama sin valor.
6	**Condicionales repetidos sobre un tipo** (*Repeated Switches*)	23–29 (zonas), 34–39 (clientes)	Cadenas `if / else if` sobre una cadena de texto	Agregar una zona o un tipo de cliente obliga a modificar `calc` (viola abierto/cerrado), justo lo que pide la Parte 8.
7	**Datos globales mutables** (*Global Data*)	2–9 `CONFIG`	Objeto global modificable desde cualquier módulo	Cualquier código puede cambiar las tarifas en tiempo de ejecución; los errores son difíciles de rastrear.
8	**Configuración inconsistente / conocimiento duplicado**	2–9 frente a 13, 35, 42, 45	Unos parámetros están en `CONFIG` y otros están quemados (`50`, `0.05`, `0.1`, `5000`)	No hay una fuente única de verdad para las reglas del negocio.
9	**Comentario desodorante** (*Comments*)	30 `// fin de semana`	El comentario explica `d > 6` en vez de usar un nombre	El comentario no se mantiene y además miente: `d > 6` excluye el sábado (DEF-02).

#	Mal olor (nombre técnico)	Línea(s)	Evidencia	Deuda técnica que genera
10	**Condicional anidado** (*Nested Conditional*) y comparación laxa	40–44 (`cup != null` + `cup == "ENVIO10"`), `==` en 23–41	Dos niveles para validar el cupón y comparaciones con conversión implícita	Oculta la rama faltante del cupón inválido (DEF-03); `==` permite coincidencias inesperadas por conversión de tipos.

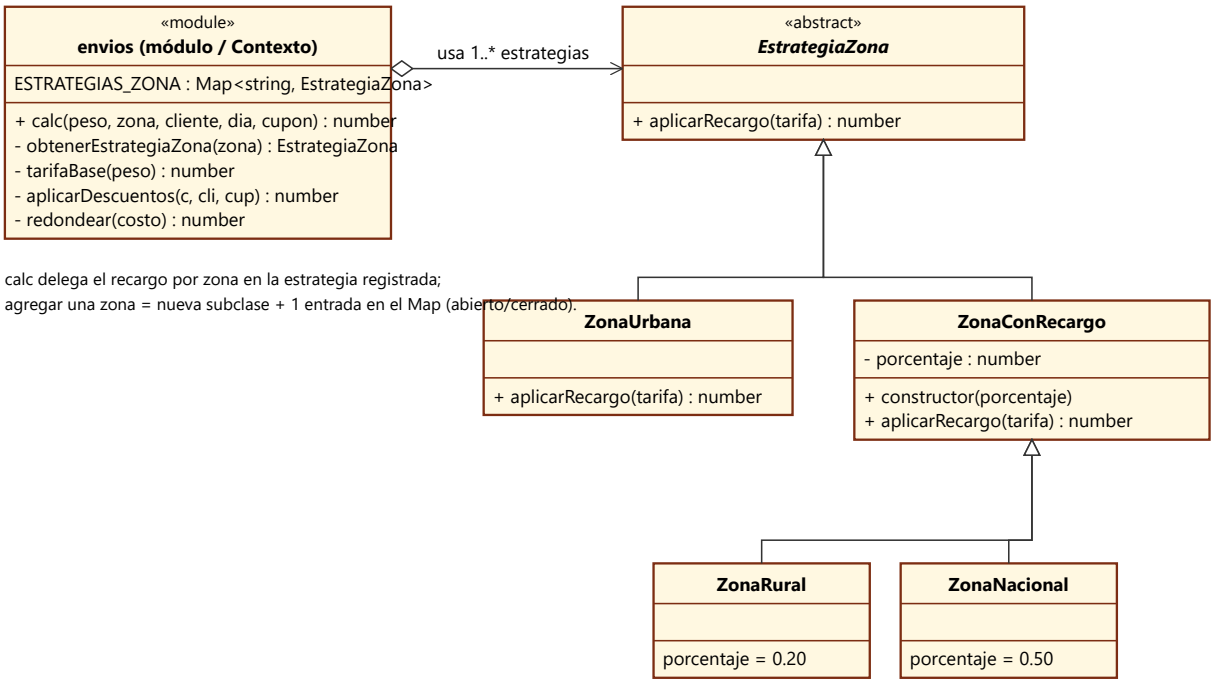
6.2 Clasificación de los cambios (ISO/IEC 14764)

Cambio	Rama	Tipo de mantenimiento	Justificación
DEF-01 a DEF-07	`fix/DEF-01` … `fix/DEF-07`	**Correctivo**	Corrigen fallas frente a la especificación.
Reemplazar números mágicos por constantes con nombre	`refactor/estrategia`	**Preventivo**	No cambia el comportamiento; reduce el riesgo de fallas futuras al modificar tarifas.
Extraer funciones (`validarPeso` , `tarifaBase` , `aplicarDescuentos` , `redondear` , …)	`refactor/estrategia`	**Preventivo**	Mejora la mantenibilidad y la capacidad de prueba (V(G) por función ≤ 4).
Patrón Estrategia en zonas	`refactor/estrategia`	**Preventivo**	Prepara el código para nuevas zonas sin modificar `calc` (abierto/cerrado).
Eliminar el argumento `flag` y el `console.log`	`refactor/estrategia`	**Perfectivo**	Mejora la interfaz y separa el cálculo de la presentación.
Validar que el peso sea numérico	`refactor/estrategia`	**Preventivo**	Evita que `"5"` (texto) se acepte por la conversión implícita de JavaScript.
Solicitud de cambio CR-01: zona insular	`feat/zona-insular`	**Perfectivo**	Nueva funcionalidad pedida por el negocio (ver Parte 8).
(Ninguno)	—	Adaptativo	No hubo cambios de plataforma ni de entorno. La traducción a JavaScript no se clasifica como mantenimiento del producto, porque conserva la lógica original.

6.3 Refactorización realizada

1. **Constantes con nombre:** `PESO_MAXIMO_KG` , `LIMITE_TRAMO_1_KG` , `LIMITE_TRAMO_2_KG` , `TARIFA_TRAMO_1` , `TARIFA_TRAMO_2` , `VALOR_KG_ADICIONAL` , `RECARGO_RURAL` , `RECARGO_NACIONAL` , `RECARGO_FIN_DE_SEMANA` , `SABADO` , `DESCUENTOS_CLIENTE` , `CUPONES` , `COSTO_MINIMO` , `MULTIPLO_REDONDEO` . El objeto global mutable `CONFIG` desaparece; las tablas se congelan con `Object.freeze` .
2. **Funciones extraídas:** `validarPeso` , `validarDia` , `obtenerEstrategiaZona` , `tarifaBase` , `esFinDeSemana` , `porcentajeCliente` , `porcentajeCupon` , `aplicarDescuentos` , `redondear` . Ahora `calc` solo coordina (V(G) = 2).
3. **Se eliminó `flag` :** `calc(peso, zona, cliente, dia, cupon = null)` . Es un cambio incompatible de la interfaz → **v2.0.0**.
4. **Patrón Estrategia en las zonas:** `EstrategiaZona` (abstracta) → `ZonaUrbana` , `ZonaConRecargo` → `ZonaRural` , `ZonaNacional` . `calc` obtiene la estrategia desde el registro `ESTRATEGIAS_ZONA` y le delega el recargo. Una zona desconocida produce un error.

Diagrama de clases UML · Patrón Estrategia aplicado a las zonas (v2.0.0)



6.4 Corrección y regresión

- Cada defecto se corrigió en su propia rama (`fix/DEF-01` … `fix/DEF-07`) y se integró con `merge --no-ff` . Después de cada integración se ejecutó toda la suite:

Después de	Pasan / total
v1.0.0 (heredado)	30 / 51
DEF-01	32 / 51
DEF-02	34 / 51
DEF-03	36 / 51
DEF-04	38 / 51
DEF-05	40 / 51
DEF-06	50 / 51
DEF-07 → **v1.0.1**	**51 / 51** ✅
Refactorización → **v2.0.0**	**62 / 62** ✅ (51 originales + 11 unitarias)

- Las **mismas** 51 pruebas se volvieron a ejecutar sin cambios después de la refactorización. La única excepción es CB15, que probaba el argumento `flag` : como ese argumento se eliminó a propósito (cambio incompatible), CB15 ahora verifica que un 6.º argumento ya no imprime nada. Capturas: `evidencias/02_corregido.png` y `evidencias/03_refactor.png` .

Evidencia: regresión en verde después de corregir (v1.0.1)

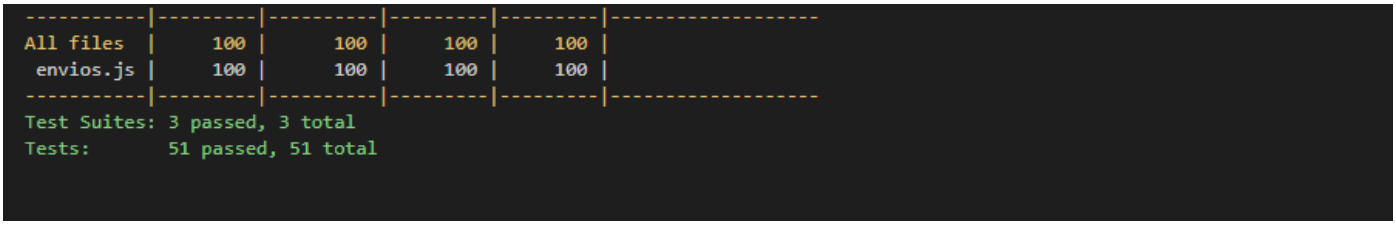
```
PS C:\parcialII> npm run test:cov # regresión tras corregir DEF-01..DEF-07

PASS tests/cajaNegra.reglas.test.js
  R4 - Recargo por zona
    √ TC16 [R4][PE] rural, 5 kg -> 9000 * 1.20 = 10800 (14 ms)
    √ TC17 [R4][PE] nacional, 5 kg -> 9000 * 1.50 = 13500 (1 ms)
    √ TC18 [R4][PE] nacional, 1 kg -> 6000 * 1.50 = 9000 (1 ms)
  R5 - Recargo de fin de semana
    √ TC19 [R5][VL] sábado (6) -> 9000 + 2000 = 11000 (1 ms)
    √ TC20 [R5][VL] domingo (7) -> 11000 (1 ms)
    √ TC21 [R5][VL] viernes (5) -> sin recargo = 9000
  R6 - Tabla de decisión cliente x cupón x fin de semana
    √ TC22 [R6][TD] frecuente, entre semana -> 8550 redondeado = 8600 (1 ms)
    √ TC23 [R6][TD] vip, entre semana -> 7650 redondeado = 7700 (1 ms)
    √ TC24 [R6][TD] normal + ENVIO10 -> 8100
    √ TC25 [R6][TD] vip + ENVIO10 -> 9000*0.85*0.90 = 6885 -> 6900
    √ TC36 [R6][TD] frecuente + ENVIO10 -> 9000*0.95*0.90 = 7695 -> 7700
    √ TC26 [R6][TD] frecuente + ENVIO10 + domingo -> 11000*0.95*0.90 = 9405 -> 9400 (1 ms)
    √ TC27 [R6][TD] vip + ENVIO10 + sábado -> 11000*0.85*0.90 = 8415 -> 8400 (1 ms)
    √ TC28 [R6][TD] frecuente + sábado -> 11000*0.95 = 10450 -> 10500
    √ TC29 [R6][TD] vip + domingo -> 11000*0.85 = 9350 -> 9400 (1 ms)
  R7 - Cupón inexistente
    √ TC30 [R7][PE] cupón 'GRATIS' lanza error (F impar) (19 ms)
  R8 - Costo mínimo y redondeo
    √ TC31 [R8][VL] 1 kg vip + ENVIO10 -> 4590 sube al mínimo 5000
    √ TC32 [R8][VL] 2 kg frecuente + ENVIO10 -> 5130 -> 5100 (1 ms)
  Casos integrales (todas las reglas)
    √ TC33 [R3-R8] 4 kg rural vip sábado ENVIO10 -> 9792 -> 9800
    √ TC34 [R3-R8] 12.5 kg nacional frecuente domingo ENVIO10 -> 15817.5 -> 15800

PASS tests/cajaBlanca.test.js
  Caminos independientes de calc
    √ CB01 camino base: 5 kg urbana normal lunes -> 9000 (16 ms)
    √ CB02 peso <= 0 verdadero -> error (20 ms)
    √ CB03 peso > 50 verdadero -> error (1 ms)
    √ CB04 tramo 1: 2 kg -> 6000
    √ CB05 tramo 3: 12 kg -> 9000 + 2*800 = 10600 (1 ms)
    √ CB06 zona rural -> 10800
    √ CB07 zona nacional -> 13500 (1 ms)
    √ CB08 ninguna zona coincide ('marte') -> error
    √ CB09 fin de semana (domingo) -> 11000 (1 ms)
    √ CB10 cliente frecuente -> 8550 -> 8600
    √ CB11 cliente vip -> 7650 -> 7700 (1 ms)
    √ CB12 cupón ENVIO10 -> 8100
    √ CB13 cupón distinto de ENVIO10 ('XYZ') -> error (1 ms)
    √ CB14 costo < 5000: 1 kg vip ENVIO10 -> 5000
    √ CB15 flag verdadero imprime el costo y retorna 9000 (2 ms)

PASS tests/cajaNegra.validaciones.test.js
  R1 - Validación del peso
    √ TC01 [R1][VL] peso = 0 lanza 'peso inválido' (35 ms)
    √ TC02 [R1][PE] peso negativo (-1) lanza error (1 ms)
    √ TC03 [R1][VL] peso = 50.01 (PMAX + 0.01) lanza error (1 ms)
    √ TC04 [R1][VL] peso = 50 (PMAX) es válido -> 41000
    √ TC05 [R1][VL] peso = 0.01 es válido -> 6000 (1 ms)
  R3 - Valores límite de los tramos
    √ TC06 [R3][VL] peso = 2.99 (L1 - 0.01) -> tramo 1 = 6000
    √ TC07 [R3][VL] peso = 3 (L1, excluido) -> tramo 2 = 9000 (1 ms)
    √ TC08 [R3][VL] peso = 3.01 (L1 + 0.01) -> 9000
    √ TC09 [R3][VL] peso = 10 (L2, incluido) -> 9000 (1 ms)
    √ TC10 [R3][VL] peso = 10.01 (L2 + 0.01) -> 9008 redondeado = 9000
    √ TC11 [R3][PE] peso = 10.5 -> 9000 + 0.5*800 = 9400 (1 ms)
    √ TC12 [R3][VL] peso = 49.99 (PMAX - 0.01) -> 40992 redondeado = 41000
    √ TC35 [R3][VL] peso = 3 rural -> 9000 * 1.20 = 10800 (1 ms)
  R2 - Validación de zona y día
    √ TC13 [R2][PE] zona inexistente 'marte' lanza error
    √ TC14 [R2][VL] día = 0 lanza error (1 ms)
    √ TC15 [R2][VL] día = 8 lanza error

-----------|---------|----------|---------|---------|-------------------
File       | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
```



Captura: npm run test:cov # regresión tras corregir DEF-01..DEF-07 — archivo completo en `evidencias/02_corregido.txt` y `.png`

Evidencia: regresión en verde después de refactorizar (v2.0.0)

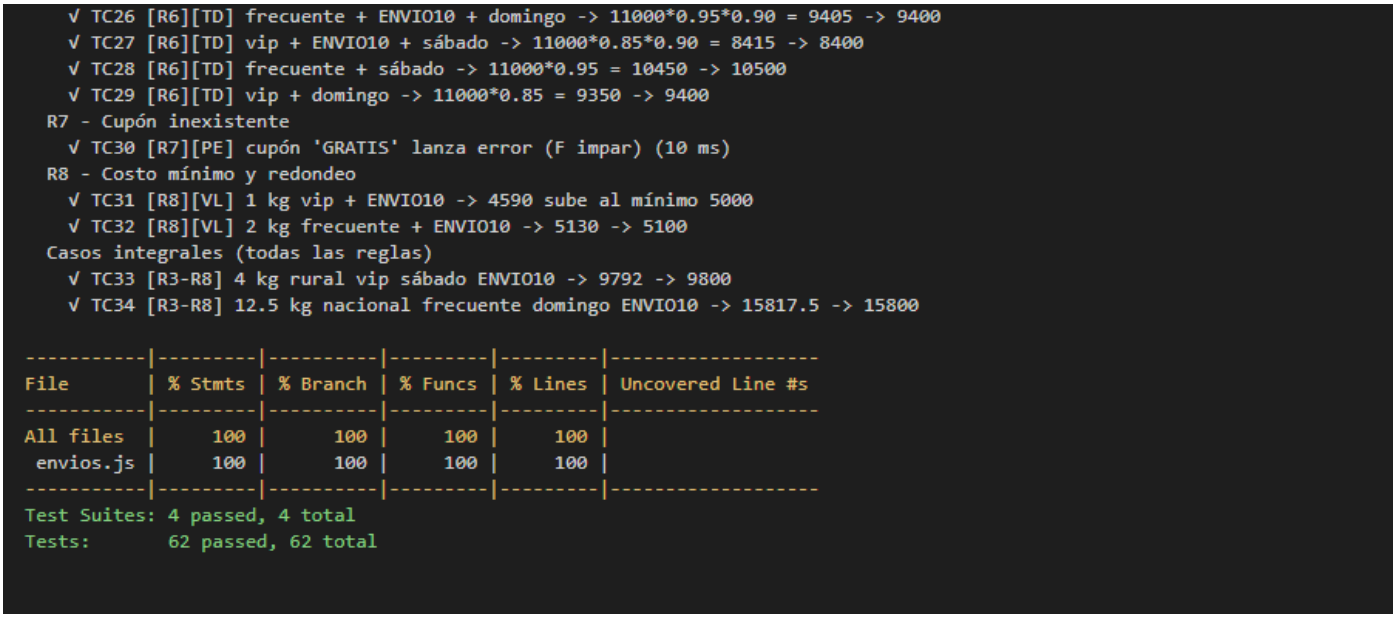
```
PS C:\parcialII> npm run test:cov # regresión tras refactorizar (v2.0.0)

PASS tests/estrategias.test.js
  Estrategias de zona
    √ UT01 la estrategia abstracta no se puede usar directamente (14 ms)
    √ UT02 ZonaUrbana no cobra recargo
    √ UT03 ZonaRural cobra 20 %
    √ UT04 ZonaNacional cobra 50 %
    √ UT05 el registro contiene las zonas válidas (1 ms)
  Funciones extraídas
    √ UT06 tarifaBase en los límites 2.99 / 3 / 10 / 12
    √ UT07 esFinDeSemana solo para 6 y 7
    √ UT08 aplicarDescuentos combina cliente y luego cupón
    √ UT09 redondear: $50 sube, $49.99 baja (1 ms)
    √ UT10 un peso que no es número ("5") lanza 'peso inválido'
    √ UT11 supuesto: un cliente no especificado se trata como normal (sin descuento) -> 9000

PASS tests/cajaNegra.validaciones.test.js
  R1 - Validación del peso
    √ TC01 [R1][VL] peso = 0 lanza 'peso inválido' (11 ms)
    √ TC02 [R1][PE] peso negativo (-1) lanza error
    √ TC03 [R1][VL] peso = 50.01 (PMAX + 0.01) lanza error
    √ TC04 [R1][VL] peso = 50 (PMAX) es válido -> 41000
    √ TC05 [R1][VL] peso = 0.01 es válido -> 6000 (1 ms)
  R3 - Valores límite de los tramos
    √ TC06 [R3][VL] peso = 2.99 (L1 - 0.01) -> tramo 1 = 6000
    √ TC07 [R3][VL] peso = 3 (L1, excluido) -> tramo 2 = 9000
    √ TC08 [R3][VL] peso = 3.01 (L1 + 0.01) -> 9000
    √ TC09 [R3][VL] peso = 10 (L2, incluido) -> 9000
    √ TC10 [R3][VL] peso = 10.01 (L2 + 0.01) -> 9008 redondeado = 9000 (1 ms)
    √ TC11 [R3][PE] peso = 10.5 -> 9000 + 0.5*800 = 9400
    √ TC12 [R3][VL] peso = 49.99 (PMAX - 0.01) -> 40992 redondeado = 41000 (4 ms)
    √ TC35 [R3][VL] peso = 3 rural -> 9000 * 1.20 = 10800
  R2 - Validación de zona y día
    √ TC13 [R2][PE] zona inexistente 'marte' lanza error
    √ TC14 [R2][VL] día = 0 lanza error (1 ms)
    √ TC15 [R2][VL] día = 8 lanza error

PASS tests/cajaBlanca.test.js
  Caminos independientes de calc
    √ CB01 camino base: 5 kg urbana normal lunes -> 9000 (1 ms)
    √ CB02 peso <= 0 verdadero -> error (10 ms)
    √ CB03 peso > 50 verdadero -> error (1 ms)
    √ CB04 tramo 1: 2 kg -> 6000
    √ CB05 tramo 3: 12 kg -> 9000 + 2*800 = 10600
    √ CB06 zona rural -> 10800
    √ CB07 zona nacional -> 13500
    √ CB08 ninguna zona coincide ('marte') -> error (1 ms)
    √ CB09 fin de semana (domingo) -> 11000
    √ CB10 cliente frecuente -> 8550 -> 8600
    √ CB11 cliente vip -> 7650 -> 7700
    √ CB12 cupón ENVIO10 -> 8100 (1 ms)
    √ CB13 cupón distinto de ENVIO10 ('XYZ') -> error
    √ CB14 costo < 5000: 1 kg vip ENVIO10 -> 5000
    √ CB15 (v2) flag eliminado: un 6.º argumento no imprime nada -> 9000 (1 ms)

PASS tests/cajaNegra.reglas.test.js
  R4 - Recargo por zona
    √ TC16 [R4][PE] rural, 5 kg -> 9000 * 1.20 = 10800
    √ TC17 [R4][PE] nacional, 5 kg -> 9000 * 1.50 = 13500
    √ TC18 [R4][PE] nacional, 1 kg -> 6000 * 1.50 = 9000 (1 ms)
  R5 - Recargo de fin de semana
    √ TC19 [R5][VL] sábado (6) -> 9000 + 2000 = 11000
    √ TC20 [R5][VL] domingo (7) -> 11000
    √ TC21 [R5][VL] viernes (5) -> sin recargo = 9000
  R6 - Tabla de decisión cliente x cupón x fin de semana
    √ TC22 [R6][TD] frecuente, entre semana -> 8550 redondeado = 8600
    √ TC23 [R6][TD] vip, entre semana -> 7650 redondeado = 7700 (1 ms)
    √ TC24 [R6][TD] normal + ENVIO10 -> 8100
    √ TC25 [R6][TD] vip + ENVIO10 -> 9000*0.85*0.90 = 6885 -> 6900
    √ TC36 [R6][TD] frecuente + ENVIO10 -> 9000*0.95*0.90 = 7695 -> 7700
```



Captura: npm run test:cov # regresión tras refactorizar (v2.0.0) — archivo completo en `evidencias/03_refactor.txt` y `.png`

Parte 6 · Comparación antes / después

Medido con ESLint (regla `complexity`, que cuenta la complejidad ciclomática de cada función) y con Jest `--coverage`.

Métrica	Antes (v1.0.0 heredado)	Corregido (v1.0.1)	Después (v2.0.0 refactorizado)
V(G) de `calc`	**15**	18	**2**
V(G) máxima de una función	15	18	**4** (`validarPeso`, `validarDia`, `porcentajeCupon`)
Número de funciones o métodos	1	1	16
V(G) total del módulo (suma)	15	18	30
Líneas físicas	54	62	154
Líneas de código (sin comentarios ni líneas en blanco)	50	57	114
Números mágicos (literales de negocio en el código)	13	17	0
Pruebas	51 (30 pasan)	51 (51 pasan)	62 (62 pasan)
Cobertura de sentencias	100 % (29/29)	100 % (33/33)	100 % (58/58)
Cobertura de ramas	100 % (30/30)	100 % (35/35)	100 % (28/28)
Cobertura de funciones	100 % (1/1)	100 % (1/1)	100 % (16/16)

Lectura:

- Corregir sin refactorizar **aumentó** la complejidad de `calc` de 15 a 18 (cada corrección agrega una decisión). La refactorización la bajó a **2**, y ninguna función supera 4: cada una se entiende y se prueba por separado.

- El número de líneas creció (54 → 154) por las constantes con nombre, las clases de la Estrategia y la documentación JSDoc. Es un intercambio deliberado: más líneas, pero cada una simple y con nombre propio.
- La complejidad total del módulo (30) es mayor que la de una sola función, pero está **repartida** en 16 unidades de V(G) 1–4. Lo que importa para el mantenimiento es la complejidad de cada unidad que hay que entender a la vez.
- La cobertura se mantuvo en el 100 % de sentencias y ramas después de refactorizar, gracias a las pruebas unitarias UT01–UT11.

Código refactorizado (envios.js v2.0.0)

```
// envios.js — cálculo del costo de envío de EnvíaYa (v2.0.0)
// Especificación personalizada: código 0182410125 (A=4 B=1 C=0 D=1 E=2 F=5)
"use strict";

// ---- Peso y tarifa base (R1, R3) ----
const PESO_MAXIMO_KG = 50;
const LIMITE_TRAMO_1_KG = 3; // D impar: el tramo 1 NO incluye el límite (peso < 3)
const LIMITE_TRAMO_2_KG = 10; // el tramo 2 sí incluye el límite (peso <= 10)
const TARIFA_TRAMO_1 = 6000;
const TARIFA_TRAMO_2 = 9000;
const VALOR_KG_ADICIONAL = 800;

// ---- Recargos (R4, R5) ----
const RECARGO_RURAL = 0.2;
const RECARGO_NACIONAL = 0.5;
const RECARGO_FIN_DE_SEMANA = 2000; // E par: se suma antes de los descuentos
const DIA_MINIMO = 1;
const DIA_MAXIMO = 7;
const SABADO = 6;

// ---- Descuentos (R6, R7) ----
const DESCUENTOS_CLIENTE = Object.freeze({ normal: 0, frecuente: 0.05, vip: 0.15 });
const CUPONES = Object.freeze({ ENVIO10: 0.1 });

// ---- Costo mínimo y redondeo (R8) ----
const COSTO_MINIMO = 5000;
const MULTIPLO_REDONDEO = 100;
const DECIMALES_TOLERANCIA = 6;

// ===== Patrón Estrategia: recargo por zona (R4) =====
class EstrategiaZona {
  aplicarRecargo(_tarifa) {
    throw new Error("EstrategiaZona es abstracta: implemente aplicarRecargo");
  }
}

class ZonaUrbana extends EstrategiaZona {
  aplicarRecargo(tarifa) {
    return tarifa;
  }
}

class ZonaConRecargo extends EstrategiaZona {
  constructor(porcentaje) {
    super();
    this.porcentaje = porcentaje;
  }

  aplicarRecargo(tarifa) {
```

```
    return tarifa + tarifa * this.porcentaje;
  }
}

class ZonaRural extends ZonaConRecargo {
  constructor() {
    super(RECARGO_RURAL);
  }
}

class ZonaNacional extends ZonaConRecargo {
  constructor() {
    super(RECARGO_NACIONAL);
  }
}

const ESTRATEGIAS_ZONA = new Map([
  ["urbana", new ZonaUrbana()],
  ["rural", new ZonaRural()],
  ["nacional", new ZonaNacional()],
]);

// ===== Funciones extraídas =====
function validarPeso(peso) {
  if (typeof peso !== "number" || !(peso > 0 && peso <= PESO_MAXIMO_KG)) {
    throw new Error("peso inválido");
  }
}

function validarDia(dia) {
  if (!Number.isInteger(dia) || dia < DIA_MINIMO || dia > DIA_MAXIMO) {
    throw new Error("día inválido");
  }
}

function obtenerEstrategiaZona(zona) {
  const estrategia = ESTRATEGIAS_ZONA.get(zona);
  if (!estrategia) throw new Error("zona inválida");
  return estrategia;
}

function tarifaBase(peso) {
  if (peso < LIMITE_TRAMO_1_KG) return TARIFA_TRAMO_1;
  if (peso <= LIMITE_TRAMO_2_KG) return TARIFA_TRAMO_2;
  return TARIFA_TRAMO_2 + (peso - LIMITE_TRAMO_2_KG) * VALOR_KG_ADICIONAL;
}

function esFinDeSemana(dia) {
  return dia >= SABADO;
}

function porcentajeCliente(cliente) {
  return Object.hasOwn(DESCUENTOS_CLIENTE, cliente) ? DESCUENTOS_CLIENTE[cliente] : 0;
}

function porcentajeCupon(cupon) {
  if (cupon === null || cupon === undefined) return 0;
  if (!Object.hasOwn(CUPONES, cupon)) throw new Error("cupón inválido");
  return CUPONES[cupon];
}

// B impar: se aplican ambos descuentos, primero el del cliente y luego el cupón
function aplicarDescuentos(costo, cliente, cupon) {
  const conDescuentoCliente = costo - costo * porcentajeCliente(cliente);
  return conDescuentoCliente - conDescuentoCliente * porcentajeCupon(cupon);
```

```
}

function redondear(costo) {
  const exacto = Number(costo.toFixed(DECIMALES_TOLERANCIA));
  return Math.floor(exacto / MULTIPLO_REDONDEO + 0.5) * MULTIPLO_REDONDEO;
}

/**
 * Calcula el costo de un envío.
 * @param {number} peso   kg, 0 < peso <= 50
 * @param {string} zona   "urbana" | "rural" | "nacional"
 * @param {string} cliente "normal" | "frecuente" | "vip"
 * @param {number} dia    1..7 (6 = sábado, 7 = domingo)
 * @param {string|null} [cupon] "ENVIO10" u omitido
 * @returns {number} costo en pesos, múltiplo de 100
 */
function calc(peso, zona, cliente, dia, cupon = null) {
  validarPeso(peso);
  validarDia(dia);
  const estrategiaZona = obtenerEstrategiaZona(zona);
  let costo = estrategiaZona.aplicarRecargo(tarifaBase(peso));
  if (esFinDeSemana(dia)) costo += RECARGO_FIN_DE_SEMANA;
  costo = aplicarDescuentos(costo, cliente, cupon);
  return redondear(Math.max(costo, COSTO_MINIMO));
}

module.exports = {
  calc,
  calcularCosto: calc,
  tarifaBase,
  esFinDeSemana,
  aplicarDescuentos,
  redondear,
  EstrategiaZona,
  ZonaUrbana,
  ZonaConRecargo,
  ZonaRural,
  ZonaNacional,
  ESTRATEGIAS_ZONA,
};
```

Parte 7 · Gestión de configuración

Parte 7 · Gestión de configuración

Repositorio: https://github.com/calo159/parcialll

7.1 Etiquetas

Etiqueta	Commit	Contenido
`v1.0.0`	código heredado	`envios.js` traducido con `CONFIG` personalizado, sin modificar
`v1.0.1`	después de integrar `fix/DEF-01` … `fix/DEF-07`	7 defectos corregidos
`v2.0.0`	después de integrar `refactor/estrategia`	refactorización, patrón Estrategia, sin `flag`
`v2.1.0`	después de integrar `feat/zona-insular`	nueva zona insular (CR-01)

Todas son etiquetas anotadas (`git tag -a`).

7.2 Ramas

Rama	Propósito	Integración
`main`	Línea principal; cada versión se etiqueta aquí	—
`test/suite`	Suite automatizada (caja negra y caja blanca) y evidencia contra el código heredado	`merge --no-ff`
`fix/DEF-01` … `fix/DEF-07`	Una rama por defecto	`merge --no-ff`, una por una, con la regresión después de cada una
`refactor/estrategia`	Constantes, funciones extraídas, Estrategia, eliminación de `flag`	`merge --no-ff`
`feat/zona-insular`	Solicitud de cambio CR-01 con pruebas primero	`merge --no-ff`

Se usó `--no-ff` para que cada integración quede como un commit de merge visible en el grafo.

7.3 Versionado semántico justificado

SemVer: **MAJOR**.MINOR.PATCH. MAJOR = cambio incompatible de la interfaz pública; MINOR = funcionalidad nueva compatible; PATCH = corrección compatible.

- **v1.0.0 → v1.0.1 (PATCH):** solo se corrigen defectos y la firma de `calc` no cambia. Aunque algunas llamadas ahora lanzan error (zona o día inválidos), esas entradas nunca fueron válidas según la especificación: corregir un comportamiento que contradice el contrato documentado es un PATCH.
- **v1.0.1 → v2.0.0 (MAJOR):** *¿quitar `flag` cambia la interfaz de la función? ¿Eso la hace major?* **Sí.** `flag` era un parámetro público de `calc`. Cualquier cliente que llamara `calc(..., true)` para registrar el costo en consola **pierde ese comportamiento**. En JavaScript la llamada no falla, pero deja de imprimir en silencio, lo que es aún más peligroso. Romper la compatibilidad de la API pública exige incrementar MAJOR. Por eso el commit usa `refactor!:` y el pie `BREAKING CHANGE:` (Conventional Commits).

- **v2.0.0 → v2.1.0 (MINOR):** la zona insular es funcionalidad nueva **compatible**: ninguna llamada válida en 2.0.0 cambia su resultado.

7.4 CHANGELOG y notas de versión

- `CHANGELOG.md` en formato *Keep a Changelog* (secciones Added / Changed / Removed / Fixed / Security, versión y fecha, enlaces de comparación).
- `RELEASE_NOTES.md` con las notas de cada versión para el usuario del módulo.

7.5 Historial

Los commits se hicieron a lo largo de la jornada, cada uno con un mensaje *Conventional Commits* (`feat`, `fix`, `test`, `docs`, `refactor`, `release`). La salida de `git log --oneline --graph --all` está en `evidencias/git_log.txt` / `.png` y en el informe.

Salida de `git log --oneline --graph --all`

```
* b732d87 (HEAD -> main, tag: v2.1.0, origin/main) release: v2.1.0 - nueva zona insular (CHANGELOG y notas de
versión)
*   5c511fc merge: integra feat/zona-insular en main
|\
| * 02aa2d8 (origin/feat/zona-insular, feat/zona-insular) feat: zona insular San Andrés con recargo del 70 %
que no acepta cupón
| * 90745be test: pruebas de la zona insular escritas primero (4 fallan, TDD en rojo)
| * cb101d7 docs(parte8): análisis de impacto de la solicitud de cambio CR-01 (zona insular)
|/
* 2bfed1c (tag: v2.0.0) release: v2.0.0 - refactorización con patrón Estrategia (cambio incompatible: sin flag)
*   9c512af merge: integra refactor/estrategia en main
|\
| * 7209539 (origin/refactor/estrategia, refactor/estrategia) test: pruebas unitarias de las estrategias de
zona y funciones extraídas
| * 09fe40f refactor!: aplica patrón Estrategia a las zonas y elimina el argumento flag
| * a6758ff refactor: extrae funciones y reemplaza números mágicos por constantes con nombre
|/
* 67014ba docs(parte6): malos olores, deuda técnica y clasificación del mantenimiento
* 291e196 (tag: v1.0.1) release: v1.0.1 - corrección de DEF-01 a DEF-07, regresión en verde
*   1104e41 merge: integra fix/DEF-07 en main
|\
| * b9cf855 (origin/fix/DEF-07, fix/DEF-07) fix(DEF-07): mensaje de error 'peso inválido' según la
especificación
|/
*   63d7041 merge: integra fix/DEF-06 en main
|\
| * 8b74055 (origin/fix/DEF-06, fix/DEF-06) fix(DEF-06): redondear al múltiplo de 100 más cercano en vez de
truncar
|/
*   9e3f774 merge: integra fix/DEF-05 en main
|\
| * 413f01b (origin/fix/DEF-05, fix/DEF-05) fix(DEF-05): un día fuera de 1..7 debe lanzar error
|/
*   844f6ad merge: integra fix/DEF-04 en main
|\
| * 9deafdb (origin/fix/DEF-04, fix/DEF-04) fix(DEF-04): una zona no válida debe lanzar error
|/
*   4c27310 merge: integra fix/DEF-03 en main
|\
| * eefe4ff (origin/fix/DEF-03, fix/DEF-03) fix(DEF-03): un cupón inexistente debe lanzar error (F impar)
|/
```

```
*   fa9c8c1 merge: integra fix/DEF-02 en main
|\
| * 70a9556 (origin/fix/DEF-02, fix/DEF-02) fix(DEF-02): el sábado (día 6) no cobraba recargo de fin de semana
|/
*   5372029 merge: integra fix/DEF-01 en main
|\
| * b228ae4 (origin/fix/DEF-01, fix/DEF-01) fix(DEF-01): tramo 1 debe excluir el límite L1 = 3 kg (D impar)
|/
* 401ec20 docs(parte5): reporte de 7 defectos con severidad, prioridad y métricas
*   ca3cefa merge: integra test/suite en main
|\
| * 2a8ddf1 (origin/test/suite, test/suite) test: evidencia de ejecución contra el código heredado (21 de 51
fallan)
| * e40996c test: casos de caja blanca, uno por camino independiente (CB01-CB15)
| * 49ce922 test: casos de caja negra de zonas, fin de semana, descuentos y redondeo (R4-R8)
| * 20505b7 test: casos de caja negra de validación (R1, R2) y valores límite (R3)
|/
* c7f5690 docs(parte4): grafo de flujo, complejidad ciclomática y caminos independientes
* ab48bd3 docs(parte3): matriz de 36 casos de caja negra con cálculos a mano
* 81ec0c7 docs(parte3): particiones de equivalencia, valores límite y tabla de decisión
* 4d68da6 docs(parte2): plan de pruebas (alcance, niveles, estrategia, riesgos, criterios)
* 7f34b46 docs(parte1): historias de usuario Dado/Cuando/Entonces y matriz de trazabilidad
* d61475b docs(parte1): parámetros personales y especificación R1-R8
* 2638a95 (tag: v1.0.0) feat: agrega código heredado envios.js (traducción fiel de envios.py) con CONFIG
personalizado
* 384627d chore: configuración inicial del proyecto (package.json, README, .gitignore)
```

```
PS C:\parcialII> git log --oneline --graph --all
* b732d87 (HEAD -> main, tag: v2.1.0, origin/main) release: v2.1.0 - nueva zona insular (CHANGELOG y notas de versión)
*   5c511fc merge: integra feat/zona-insular en main
|\
| * 02aa2d8 (origin/feat/zona-insular, feat/zona-insular) feat: zona insular San Andrés con recargo del 70 % que no acepta cupón
| * 90745be test: pruebas de la zona insular escritas primero (4 fallan, TDD en rojo)
| * cb101d7 docs(parte8): análisis de impacto de la solicitud de cambio CR-01 (zona insular)
|/
* 2bfed1c (tag: v2.0.0) release: v2.0.0 - refactorización con patrón Estrategia (cambio incompatible: sin flag)
*   9c512af merge: integra refactor/estrategia en main
|\
| * 7209539 (origin/refactor/estrategia, refactor/estrategia) test: pruebas unitarias de las estrategias de zona y funciones extraídas
| * 09fe40f refactor!: aplica patrón Estrategia a las zonas y elimina el argumento flag
| * a6758ff refactor: extrae funciones y reemplaza números mágicos por constantes con nombre
|/
* 67014ba docs(parte6): malos olores, deuda técnica y clasificación del mantenimiento
* 291e196 (tag: v1.0.1) release: v1.0.1 - corrección de DEF-01 a DEF-07, regresión en verde
*   1104e41 merge: integra fix/DEF-07 en main
|\
| * b9cf855 (origin/fix/DEF-07, fix/DEF-07) fix(DEF-07): mensaje de error 'peso inválido' según la especificación
|/
*   63d7041 merge: integra fix/DEF-06 en main
|\
| * 8b74055 (origin/fix/DEF-06, fix/DEF-06) fix(DEF-06): redondear al múltiplo de 100 más cercano en vez de truncar
|/
*   9e3f774 merge: integra fix/DEF-05 en main
|\
| * 413f01b (origin/fix/DEF-05, fix/DEF-05) fix(DEF-05): un día fuera de 1..7 debe lanzar error
|/
*   844f6ad merge: integra fix/DEF-04 en main
|\
| * 9deafdb (origin/fix/DEF-04, fix/DEF-04) fix(DEF-04): una zona no válida debe lanzar error
|/
*   4c27310 merge: integra fix/DEF-03 en main
|\
| * eefe4ff (origin/fix/DEF-03, fix/DEF-03) fix(DEF-03): un cupón inexistente debe lanzar error (F impar)
|/
*   fa9c8c1 merge: integra fix/DEF-02 en main
|\
| * 70a9556 (origin/fix/DEF-02, fix/DEF-02) fix(DEF-02): el sábado (día 6) no cobraba recargo de fin de semana
|/
*   5372029 merge: integra fix/DEF-01 en main
|\
| * b228ae4 (origin/fix/DEF-01, fix/DEF-01) fix(DEF-01): tramo 1 debe excluir el límite L1 = 3 kg (D impar)
|/
* 401ec20 docs(parte5): reporte de 7 defectos con severidad, prioridad y métricas
*   ca3cefa merge: integra test/suite en main
|\
| * 2a8ddf1 (origin/test/suite, test/suite) test: evidencia de ejecución contra el código heredado (21 de 51 fallan)
| * e40996c test: casos de caja blanca, uno por camino independiente (CB01-CB15)
| * 49ce922 test: casos de caja negra de zonas, fin de semana, descuentos y redondeo (R4-R8)
| * 20505b7 test: casos de caja negra de validación (R1, R2) y valores límite (R3)
|/
* c7f5690 docs(parte4): grafo de flujo, complejidad ciclomática y caminos independientes
* ab48bd3 docs(parte3): matriz de 36 casos de caja negra con cálculos a mano
* 81ec0c7 docs(parte3): particiones de equivalencia, valores límite y tabla de decisión
* 4d68da6 docs(parte2): plan de pruebas (alcance, niveles, estrategia, riesgos, criterios)
* 7f34b46 docs(parte1): historias de usuario Dado/Cuando/Entonces y matriz de trazabilidad
* d61475b docs(parte1): parámetros personales y especificación R1-R8
* 2638a95 (tag: v1.0.0) feat: agrega código heredado envios.js (traducción fiel de envios.py) con CONFIG personalizado
* 384627d chore: configuración inicial del proyecto (package.json, README, .gitignore)
```

Captura de git log --oneline --graph --all

CHANGELOG.md

Changelog

Todos los cambios notables de este proyecto se documentan en este archivo.

El formato se basa en Keep a Changelog y el proyecto usa Versionado Semántico.

[Unreleased]

2.1.0 - 2026-11-11

Added

- CR-01: nueva zona `insular` (San Andrés) con un recargo del 70 % (estrategia `ZonaInsular`).
- Los envíos insulares no aceptan cupón: si se envía un cupón con la zona insular, se lanza el error "los envíos insulares no aceptan cupón".
- Pruebas INS01–INS06, escritas antes de la implementación.

2.0.0 - 2026-11-11

Changed

- **BREAKING:** `calc(peso, zona, cliente, dia, cupon)` ya no recibe el argumento `flag` ni imprime el costo en consola. Quien necesite mostrar el costo debe hacerlo con el valor retornado.
- Patrón Estrategia para el recargo por zona: `EstrategiaZona`, `ZonaUrbana`, `ZonaConRecargo`, `ZonaRural`, `ZonaNacional`.
- Se extraen funciones (`validarPeso`, `validarDia`, `tarifaBase`, `esFinDeSemana`, `aplicarDescuentos`, `redondear`, …); la complejidad ciclomática de `calc` baja de 18 a 2.
- Los números mágicos se reemplazan por constantes con nombre.

Added

- Alias `calcularCosto` de `calc`.
- 11 pruebas unitarias de las estrategias y de las funciones extraídas (UT01–UT11).

Removed

- Argumento `flag` de `calc`.
- Objeto global mutable `CONFIG` (reemplazado por constantes inmutables).

Security

- El peso debe ser un número: un texto como `"5"` ya no se acepta por conversión implícita.

1.0.1 - 2026-11-11

Fixed

- DEF-01: el tramo 1 ya no incluye el límite L1 = 3 kg; un paquete de 3 kg cuesta $9.000 (R3).
- DEF-02: el sábado (día 6) ahora cobra el recargo de fin de semana de $2.000 (R5).
- DEF-03: un cupón inexistente ahora lanza el error "cupón inválido" en vez de ignorarse (R7).
- DEF-04: una zona distinta de urbana, rural o nacional ahora lanza el error "zona inválida" (R2).
- DEF-05: un día fuera del rango 1..7 ahora lanza el error "día inválido" (R2).
- DEF-06: el costo se redondea al múltiplo de $100 más cercano ($50 sube) en lugar de truncarse (R8).
- DEF-07: el mensaje de error del peso ahora es "peso inválido", como dice la especificación (R1).

1.0.0 - 2026-11-11

Added

- Código heredado `envios.js` (traducción fiel de `envios.py`) con `CONFIG` personalizado para el código 0182410125: L1 = 3, L2 = 10, rural 20 %, nacional 50 %, VIP 15 %, fin de semana $2.000.

Notas de versión

v2.1.0 — Zona insular (11-nov-2026)

Novedades

- Ahora se pueden cotizar envíos a **San Andrés**: `calc(peso, "insular", cliente, dia)`. La zona insular tiene un recargo del **70 %** sobre la tarifa base.
- Los envíos insulares **no aceptan cupón**. Si se envía uno, se lanza el error *"los envíos insulares no aceptan cupón"*.

Compatibilidad: total con v2.0.0. Actualización recomendada.

---

v2.0.0 — Código mantenible (11-nov-2026)

⚠️ Cambio incompatible

- `calc` **ya no recibe el argumento** `flag` y no imprime nada en consola. *Migración:* reemplace `calc(p, z, c, d, cup, true)` por `const costo = calc(p, z, c, d, cup); console.log("costo:", costo);`

Mejoras internas

- Patrón Estrategia para las zonas; constantes con nombre; funciones pequeñas (complejidad ciclomática de `calc`: 18 → 2).
- 62 pruebas automatizadas con 100 % de cobertura de sentencias y de ramas.

---

v1.0.1 — Correcciones (11-nov-2026)

Corrige 7 defectos del cálculo heredado:

Defecto	Antes	Ahora
DEF-01	3 kg costaba $6.000	3 kg cuesta $9.000 (tramo 2)
DEF-02	el sábado no tenía recargo	sábado y domingo: +$2.000
DEF-03	un cupón falso se ignoraba	error "cupón inválido"
DEF-04	una zona desconocida se cobraba como urbana	error "zona inválida"
DEF-05	se aceptaban días 0, 8, …	error "día inválido"
DEF-06	el costo se truncaba ($8.550 → $8.500)	se redondea al múltiplo de $100 más cercano ($8.550 → $8.600)
DEF-07	mensaje "peso invalido"	mensaje "peso inválido"

Compatibilidad: misma firma de función. Las cotizaciones de 3 kg, de sábado y con redondeo cambian porque antes eran incorrectas.

Parte 8 · Solicitud de cambio y análisis de impacto

Parte 8 · Solicitud de cambio y análisis de impacto

(A + F) mod 3 = (4 + 5) mod 3 = 9 mod 3 = 0 → solicitud de cambio **CR-01**:

> Nueva zona **insular** (San Andrés) con un recargo del **70 %**. Los envíos insulares **no aceptan cupón**.

8.1 Nuevo requisito y nueva historia

- **R4 (modificado):** recargo por zona. Rural +20 %, nacional +50 %, **insular +70 %**, urbana sin recargo.
- **R2 (modificado):** las zonas válidas ahora son urbana, rural, nacional e **insular**.
- **R9 (nuevo):** si la zona es insular y se envía cualquier cupón, se lanza el error "los envíos insulares no aceptan cupón". Decisión de diseño: como en mi especificación un cupón inexistente es error (F impar), un cupón no permitido también es **error**, no se ignora en silencio.

HU-04 · Envíos a San Andrés. Como cliente de San Andrés **quiero** cotizar envíos a la isla **para** recibir paquetes del continente.

#	Dado	Cuando	Entonces
CA-4.1	un paquete de 5 kg a zona insular, cliente normal, lunes	calculo el costo	9.000 × 1,70 = **$15.300**
CA-4.2	un envío insular con el cupón ENVIO10	calculo el costo	error "los envíos insulares no aceptan cupón"
CA-4.3	un envío insular VIP de 12 kg un sábado	calculo el costo	(10.600 × 1,70 + 2.000) × 0,85 = 17.017 → **$17.000**

8.2 Análisis de impacto (con la matriz de trazabilidad)

Elemento	¿Se afecta?	Detalle
R2 zonas válidas	Sí	"insular" deja de ser inválida
R4 recargo por zona	Sí	Nueva estrategia con 70 %
R6 / R7 cupón	Sí	El cupón depende ahora de la zona (nueva R9)
R1, R3, R5, R8	No	La tarifa base, el fin de semana, el mínimo y el redondeo se aplican igual a la zona insular
HU-01	Sí	El criterio CA-1.4 cambia: "marte" sigue siendo inválida, "insular" no
HU-02	Sí	Nueva zona en el cálculo con recargo
HU-03	Sí (parcial)	Los descuentos de cliente se aplican; el cupón queda prohibido para la zona insular
HU-04	Nueva	Historia de la solicitud de cambio

Elemento	¿Se afecta?	Detalle
Casos TC13, CB08	Revisar	Siguen válidos ("marte" sigue siendo error); no se usa "insular" como zona inválida en ninguna prueba previa
Casos TC16, TC17, TC24, TC30	Regresión	Deben seguir pasando: las otras zonas aceptan cupón (INS06)
Código `ESTRATEGIAS_ZONA`	Sí	Se registra `["insular", new ZonaInsular()]`
Código `EstrategiaZona`	Sí	Nueva propiedad `aceptaCupon` (por defecto `true`)
Código clase nueva `ZonaInsular`	Nueva	`extends ZonaConRecargo` con 70 % y `aceptaCupon = false`
Código `calc`	Sí (1 línea)	Valida el cupón contra `estrategiaZona.aceptaCupon`
`tarifaBase`, `aplicarDescuentos`, `redondear`	No	Sin cambios
Documentación	Sí	CHANGELOG, notas de versión, UML (nueva subclase)

Gracias al patrón Estrategia, el cambio no toca la lógica de cálculo de las otras zonas: se agrega una clase y una entrada en el registro. En el código heredado habría sido necesario modificar la cadena de `if / else if` de `calc` y anidar otra condición en el bloque del cupón.

8.3 Tipo de mantenimiento

Perfectivo: agrega funcionalidad nueva pedida por el negocio, sin corregir fallas (correctivo) ni responder a un cambio de entorno (adaptativo).

8.4 Implementación con pruebas primero (TDD)

1. Rama `feat/zona-insular` creada desde `main` (v2.0.0).
2. **Rojo:** se escribieron `tests/insular.test.js` (INS01–INS06) **antes** del código. Resultado: **4 fallan** (INS01–INS04, porque "insular" todavía era una zona inválida); INS05 e INS06 pasan (cupón inexistente y regresión de la zona rural). Captura: `evidencias/04_insular_rojo.png`.
3. **Verde:** se implementó `ZonaInsular`, la propiedad `aceptaCupon` y la validación en `calc`. Resultado: **68 / 68 pasan**, con 100 % de sentencias y de ramas. Captura: `evidencias/05_insular_verde.png`.
4. Integración en `main` con `merge --no-ff`, CHANGELOG actualizado y etiqueta **v2.1.0**.

Caso	Entradas	Cálculo a mano	Esperado
INS01	(5, insular, normal, 1)	9.000 × 1,70	$15.300
INS02	(5, insular, normal, 1, ENVIO10)	cupón en zona insular	error
INS03	(12, insular, vip, 6)	(9.000 + 2 × 800) × 1,70 = 18.020; + 2.000 = 20.020; × 0,85 = 17.017	$17.000
INS04	(1, insular, frecuente, 7)	6.000 × 1,70 = 10.200; + 2.000 = 12.200; × 0,95 = 11.590	$11.600
INS05	(5, insular, normal, 1, XYZ)	cupón no permitido en zona insular	error
INS06	(5, rural, normal, 1, ENVIO10)	10.800 × 0,90 = 9.720	$9.700

8.5 Versión

Se agrega funcionalidad **compatible** con versiones anteriores: todas las llamadas válidas en v2.0.0 dan el mismo resultado en v2.1.0. Según SemVer, esto es un cambio **MINOR** → **v2.1.0**.

Evidencia: pruebas escritas primero (rojo)

```
PS C:\parcialII> npm run test:cov # pruebas de zona insular escritas primero (rojo)

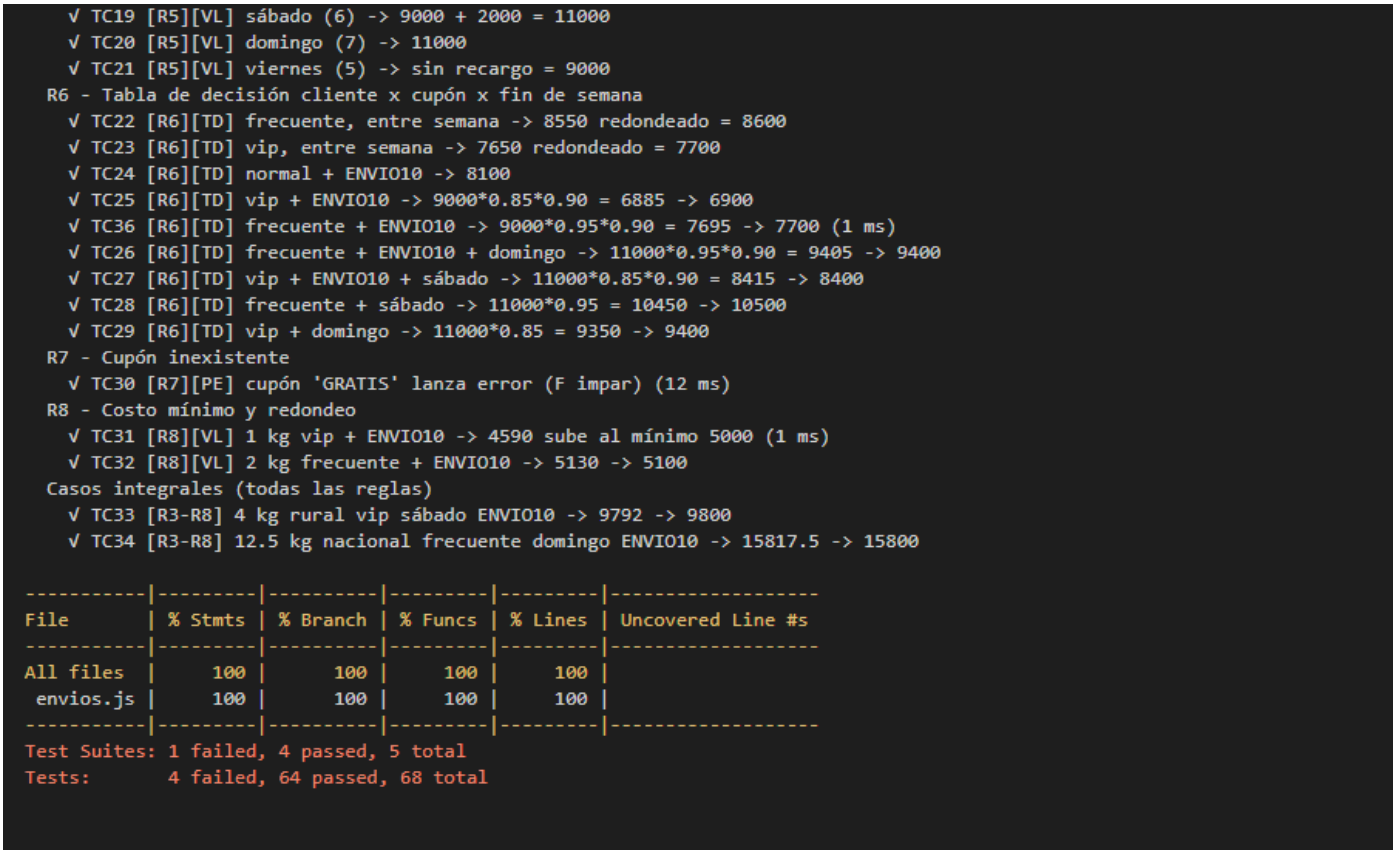
FAIL tests/insular.test.js
  CR-01 Zona insular (San Andrés)
    x INS01 5 kg insular normal lunes -> 9000 * 1.70 = 15300 (1 ms)
    x INS02 insular con cupón ENVIO10 lanza error (11 ms)
    x INS03 12 kg insular vip sábado -> (10600*1.70 + 2000)*0.85 = 17017 -> 17000
    x INS04 1 kg insular frecuente domingo -> (6000*1.70 + 2000)*0.95 = 11590 -> 11600
    √ INS05 insular con cupón inexistente también lanza error (1 ms)
    √ INS06 regresión: la zona rural sigue aceptando cupón -> 10800*0.90 = 9720 -> 9700 (1 ms)

PASS tests/estrategias.test.js
  Estrategias de zona
    √ UT01 la estrategia abstracta no se puede usar directamente (12 ms)
    √ UT02 ZonaUrbana no cobra recargo
    √ UT03 ZonaRural cobra 20 %
    √ UT04 ZonaNacional cobra 50 %
    √ UT05 el registro contiene las zonas válidas
  Funciones extraídas
    √ UT06 tarifaBase en los límites 2.99 / 3 / 10 / 12
    √ UT07 esFinDeSemana solo para 6 y 7
    √ UT08 aplicarDescuentos combina cliente y luego cupón
    √ UT09 redondear: $50 sube, $49.99 baja
    √ UT10 un peso que no es número ("5") lanza 'peso inválido'
    √ UT11 supuesto: un cliente no especificado se trata como normal (sin descuento) -> 9000 (1 ms)

PASS tests/cajaBlanca.test.js
  Caminos independientes de calc
    √ CB01 camino base: 5 kg urbana normal lunes -> 9000 (1 ms)
    √ CB02 peso <= 0 verdadero -> error (13 ms)
    √ CB03 peso > 50 verdadero -> error
    √ CB04 tramo 1: 2 kg -> 6000
    √ CB05 tramo 3: 12 kg -> 9000 + 2*800 = 10600 (1 ms)
    √ CB06 zona rural -> 10800
    √ CB07 zona nacional -> 13500
    √ CB08 ninguna zona coincide ('marte') -> error (1 ms)
    √ CB09 fin de semana (domingo) -> 11000
    √ CB10 cliente frecuente -> 8550 -> 8600
    √ CB11 cliente vip -> 7650 -> 7700
    √ CB12 cupón ENVIO10 -> 8100
    √ CB13 cupón distinto de ENVIO10 ('XYZ') -> error
    √ CB14 costo < 5000: 1 kg vip ENVIO10 -> 5000
    √ CB15 (v2) flag eliminado: un 6.º argumento no imprime nada -> 9000 (1 ms)

PASS tests/cajaNegra.validaciones.test.js
  R1 - Validación del peso
    √ TC01 [R1][VL] peso = 0 lanza 'peso inválido' (12 ms)
    √ TC02 [R1][PE] peso negativo (-1) lanza error (1 ms)
    √ TC03 [R1][VL] peso = 50.01 (PMAX + 0.01) lanza error
    √ TC04 [R1][VL] peso = 50 (PMAX) es válido -> 41000
    √ TC05 [R1][VL] peso = 0.01 es válido -> 6000 (1 ms)
  R3 - Valores límite de los tramos
    √ TC06 [R3][VL] peso = 2.99 (L1 - 0.01) -> tramo 1 = 6000
    √ TC07 [R3][VL] peso = 3 (L1, excluido) -> tramo 2 = 9000
    √ TC08 [R3][VL] peso = 3.01 (L1 + 0.01) -> 9000
    √ TC09 [R3][VL] peso = 10 (L2, incluido) -> 9000
    √ TC10 [R3][VL] peso = 10.01 (L2 + 0.01) -> 9008 redondeado = 9000
    √ TC11 [R3][PE] peso = 10.5 -> 9000 + 0.5*800 = 9400
    √ TC12 [R3][VL] peso = 49.99 (PMAX - 0.01) -> 40992 redondeado = 41000
    √ TC35 [R3][VL] peso = 3 rural -> 9000 * 1.20 = 10800
  R2 - Validación de zona y día
    √ TC13 [R2][PE] zona inexistente 'marte' lanza error
    √ TC14 [R2][VL] día = 0 lanza error (1 ms)
    √ TC15 [R2][VL] día = 8 lanza error

PASS tests/cajaNegra.reglas.test.js
  R4 - Recargo por zona
    √ TC16 [R4][PE] rural, 5 kg -> 9000 * 1.20 = 10800 (1 ms)
    √ TC17 [R4][PE] nacional, 5 kg -> 9000 * 1.50 = 13500
    √ TC18 [R4][PE] nacional, 1 kg -> 6000 * 1.50 = 9000
  R5 - Recargo de fin de semana
```



Captura: npm run test:cov # pruebas de zona insular escritas primero (rojo) — archivo completo en `evidencias/04_insular_rojo.txt` y `.png`

Evidencia: implementación (verde)

```
PS C:\parcialII> npm run test:cov # zona insular implementada (verde)

PASS tests/insular.test.js
  CR-01 Zona insular (San Andrés)
    √ INS01 5 kg insular normal lunes -> 9000 * 1.70 = 15300 (2 ms)
    √ INS02 insular con cupón ENVIO10 lanza error (17 ms)
    √ INS03 12 kg insular vip sábado -> (10600*1.70 + 2000)*0.85 = 17017 -> 17000
    √ INS04 1 kg insular frecuente domingo -> (6000*1.70 + 2000)*0.95 = 11590 -> 11600
    √ INS05 insular con cupón inexistente también lanza error (2 ms)
    √ INS06 regresión: la zona rural sigue aceptando cupón -> 10800*0.90 = 9720 -> 9700 (1 ms)

PASS tests/cajaBlanca.test.js
  Caminos independientes de calc
    √ CB01 camino base: 5 kg urbana normal lunes -> 9000 (1 ms)
    √ CB02 peso <= 0 verdadero -> error (20 ms)
    √ CB03 peso > 50 verdadero -> error (1 ms)
    √ CB04 tramo 1: 2 kg -> 6000
    √ CB05 tramo 3: 12 kg -> 9000 + 2*800 = 10600
    √ CB06 zona rural -> 10800
    √ CB07 zona nacional -> 13500
    √ CB08 ninguna zona coincide ('marte') -> error (1 ms)
    √ CB09 fin de semana (domingo) -> 11000
    √ CB10 cliente frecuente -> 8550 -> 8600 (1 ms)
    √ CB11 cliente vip -> 7650 -> 7700
    √ CB12 cupón ENVIO10 -> 8100
    √ CB13 cupón distinto de ENVIO10 ('XYZ') -> error (6 ms)
    √ CB14 costo < 5000: 1 kg vip ENVIO10 -> 5000
    √ CB15 (v2) flag eliminado: un 6.º argumento no imprime nada -> 9000 (1 ms)

PASS tests/cajaNegra.validaciones.test.js
  R1 - Validación del peso
    √ TC01 [R1][VL] peso = 0 lanza 'peso inválido' (13 ms)
    √ TC02 [R1][PE] peso negativo (-1) lanza error (1 ms)
    √ TC03 [R1][VL] peso = 50.01 (PMAX + 0.01) lanza error
    √ TC04 [R1][VL] peso = 50 (PMAX) es válido -> 41000
    √ TC05 [R1][VL] peso = 0.01 es válido -> 6000 (1 ms)
  R3 - Valores límite de los tramos
    √ TC06 [R3][VL] peso = 2.99 (L1 - 0.01) -> tramo 1 = 6000
    √ TC07 [R3][VL] peso = 3 (L1, excluido) -> tramo 2 = 9000
    √ TC08 [R3][VL] peso = 3.01 (L1 + 0.01) -> 9000
    √ TC09 [R3][VL] peso = 10 (L2, incluido) -> 9000
    √ TC10 [R3][VL] peso = 10.01 (L2 + 0.01) -> 9008 redondeado = 9000
    √ TC11 [R3][PE] peso = 10.5 -> 9000 + 0.5*800 = 9400 (1 ms)
    √ TC12 [R3][VL] peso = 49.99 (PMAX - 0.01) -> 40992 redondeado = 41000
    √ TC35 [R3][VL] peso = 3 rural -> 9000 * 1.20 = 10800
  R2 - Validación de zona y día
    √ TC13 [R2][PE] zona inexistente 'marte' lanza error
    √ TC14 [R2][VL] día = 0 lanza error (1 ms)
    √ TC15 [R2][VL] día = 8 lanza error

PASS tests/estrategias.test.js
  Estrategias de zona
    √ UT01 la estrategia abstracta no se puede usar directamente (16 ms)
    √ UT02 ZonaUrbana no cobra recargo
    √ UT03 ZonaRural cobra 20 % (1 ms)
    √ UT04 ZonaNacional cobra 50 %
    √ UT05 el registro contiene las zonas válidas
  Funciones extraídas
    √ UT06 tarifaBase en los límites 2.99 / 3 / 10 / 12 (1 ms)
    √ UT07 esFinDeSemana solo para 6 y 7
    √ UT08 aplicarDescuentos combina cliente y luego cupón
    √ UT09 redondear: $50 sube, $49.99 baja (1 ms)
    √ UT10 un peso que no es número ("5") lanza 'peso inválido'
    √ UT11 supuesto: un cliente no especificado se trata como normal (sin descuento) -> 9000

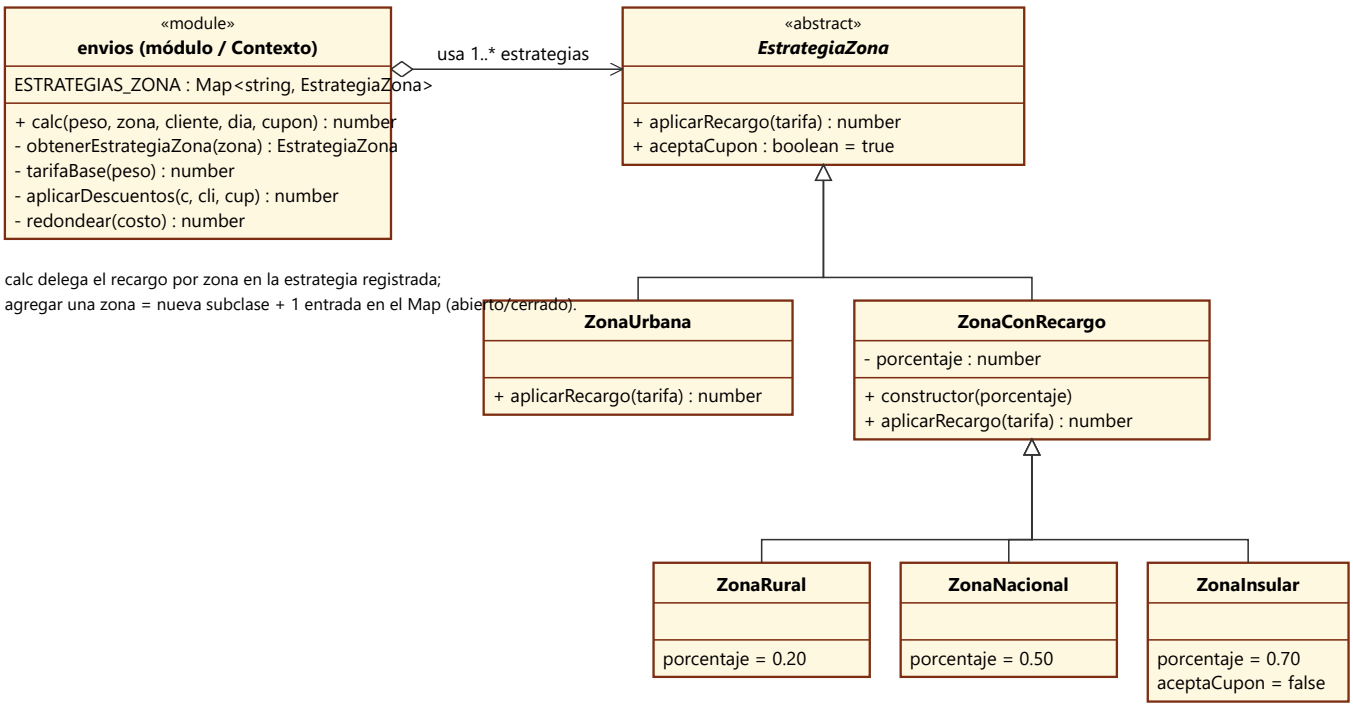
PASS tests/cajaNegra.reglas.test.js
  R4 - Recargo por zona
    √ TC16 [R4][PE] rural, 5 kg -> 9000 * 1.20 = 10800
    √ TC17 [R4][PE] nacional, 5 kg -> 9000 * 1.50 = 13500
    √ TC18 [R4][PE] nacional, 1 kg -> 6000 * 1.50 = 9000 (1 ms)
  R5 - Recargo de fin de semana
```

```
    √ TC19 [R5][VL] sábado (6) -> 9000 + 2000 = 11000
    √ TC20 [R5][VL] domingo (7) -> 11000
    √ TC21 [R5][VL] viernes (5) -> sin recargo = 9000
  R6 - Tabla de decisión cliente x cupón x fin de semana
    √ TC22 [R6][TD] frecuente, entre semana -> 8550 redondeado = 8600
    √ TC23 [R6][TD] vip, entre semana -> 7650 redondeado = 7700 (1 ms)
    √ TC24 [R6][TD] normal + ENVIO10 -> 8100
    √ TC25 [R6][TD] vip + ENVIO10 -> 9000*0.85*0.90 = 6885 -> 6900
    √ TC36 [R6][TD] frecuente + ENVIO10 -> 9000*0.95*0.90 = 7695 -> 7700
    √ TC26 [R6][TD] frecuente + ENVIO10 + domingo -> 11000*0.95*0.90 = 9405 -> 9400
    √ TC27 [R6][TD] vip + ENVIO10 + sábado -> 11000*0.85*0.90 = 8415 -> 8400
    √ TC28 [R6][TD] frecuente + sábado -> 11000*0.95 = 10450 -> 10500 (4 ms)
    √ TC29 [R6][TD] vip + domingo -> 11000*0.85 = 9350 -> 9400
  R7 - Cupón inexistente
    √ TC30 [R7][PE] cupón 'GRATIS' lanza error (F impar) (12 ms)
  R8 - Costo mínimo y redondeo
    √ TC31 [R8][VL] 1 kg vip + ENVIO10 -> 4590 sube al mínimo 5000
    √ TC32 [R8][VL] 2 kg frecuente + ENVIO10 -> 5130 -> 5100
  Casos integrales (todas las reglas)
    √ TC33 [R3-R8] 4 kg rural vip sábado ENVIO10 -> 9792 -> 9800
    √ TC34 [R3-R8] 12.5 kg nacional frecuente domingo ENVIO10 -> 15817.5 -> 15800 (1 ms)

-----------|---------|----------|---------|---------|-------------------
File       | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------|---------|----------|---------|---------|-------------------
All files  |     100 |      100 |     100 |     100 |
 envios.js |     100 |      100 |     100 |     100 |
-----------|---------|----------|---------|---------|-------------------
Test Suites: 5 passed, 5 total
Tests:       68 passed, 68 total
```

Captura: npm run test:cov # zona insular implementada (verde) — archivo completo en `evidencias/05_insular_verde.txt` y `.png`

Diagrama de clases UML · Patrón Estrategia aplicado a las zonas (v2.1.0)



UML final con la nueva estrategia ZonaInsular (v2.1.0)

Código final (envios.js v2.1.0)

```
// envios.js — cálculo del costo de envío de EnvíaYa (v2.1.0)
// Especificación personalizada: código 0182410125 (A=4 B=1 C=0 D=1 E=2 F=5)
"use strict";

// ---- Peso y tarifa base (R1, R3) ----
const PESO_MAXIMO_KG = 50;
const LIMITE_TRAMO_1_KG = 3; // D impar: el tramo 1 NO incluye el límite (peso < 3)
const LIMITE_TRAMO_2_KG = 10; // el tramo 2 sí incluye el límite (peso <= 10)
const TARIFA_TRAMO_1 = 6000;
const TARIFA_TRAMO_2 = 9000;
const VALOR_KG_ADICIONAL = 800;

// ---- Recargos (R4, R5) ----
const RECARGO_RURAL = 0.2;
const RECARGO_NACIONAL = 0.5;
const RECARGO_INSULAR = 0.7; // CR-01: San Andrés
const RECARGO_FIN_DE_SEMANA = 2000; // E par: se suma antes de los descuentos
const DIA_MINIMO = 1;
const DIA_MAXIMO = 7;
const SABADO = 6;

// ---- Descuentos (R6, R7) ----
const DESCUENTOS_CLIENTE = Object.freeze({ normal: 0, frecuente: 0.05, vip: 0.15 });
const CUPONES = Object.freeze({ ENVIO10: 0.1 });

// ---- Costo mínimo y redondeo (R8) ----
const COSTO_MINIMO = 5000;
const MULTIPLO_REDONDEO = 100;
const DECIMALES_TOLERANCIA = 6;

// ===== Patrón Estrategia: recargo por zona (R4) =====
class EstrategiaZona {
  aplicarRecargo(_tarifa) {
    throw new Error("EstrategiaZona es abstracta: implemente aplicarRecargo");
  }

  get aceptaCupon() {
    return true;
  }
}

class ZonaUrbana extends EstrategiaZona {
  aplicarRecargo(tarifa) {
    return tarifa;
  }
}

class ZonaConRecargo extends EstrategiaZona {
  constructor(porcentaje) {
    super();
    this.porcentaje = porcentaje;
  }

  aplicarRecargo(tarifa) {
    return tarifa + tarifa * this.porcentaje;
  }
}

class ZonaRural extends ZonaConRecargo {
  constructor() {
    super(RECARGO_RURAL);
  }
}
```

```
class ZonaNacional extends ZonaConRecargo {
  constructor() {
    super(RECARGO_NACIONAL);
  }
}

// CR-01: zona insular con recargo del 70 %; no acepta cupones
class ZonaInsular extends ZonaConRecargo {
  constructor() {
    super(RECARGO_INSULAR);
  }

  get aceptaCupon() {
    return false;
  }
}

const ESTRATEGIAS_ZONA = new Map([
  ["urbana", new ZonaUrbana()],
  ["rural", new ZonaRural()],
  ["nacional", new ZonaNacional()],
  ["insular", new ZonaInsular()],
]);

// ===== Funciones extraídas =====
function validarPeso(peso) {
  if (typeof peso !== "number" || !(peso > 0 && peso <= PESO_MAXIMO_KG)) {
    throw new Error("peso inválido");
  }
}

function validarDia(dia) {
  if (!Number.isInteger(dia) || dia < DIA_MINIMO || dia > DIA_MAXIMO) {
    throw new Error("día inválido");
  }
}

function obtenerEstrategiaZona(zona) {
  const estrategia = ESTRATEGIAS_ZONA.get(zona);
  if (!estrategia) throw new Error("zona inválida");
  return estrategia;
}

function tarifaBase(peso) {
  if (peso < LIMITE_TRAMO_1_KG) return TARIFA_TRAMO_1;
  if (peso <= LIMITE_TRAMO_2_KG) return TARIFA_TRAMO_2;
  return TARIFA_TRAMO_2 + (peso - LIMITE_TRAMO_2_KG) * VALOR_KG_ADICIONAL;
}

function esFinDeSemana(dia) {
  return dia >= SABADO;
}

function porcentajeCliente(cliente) {
  return Object.hasOwn(DESCUENTOS_CLIENTE, cliente) ? DESCUENTOS_CLIENTE[cliente] : 0;
}

function porcentajeCupon(cupon) {
  if (cupon === null || cupon === undefined) return 0;
  if (!Object.hasOwn(CUPONES, cupon)) throw new Error("cupón inválido");
  return CUPONES[cupon];
}

// B impar: se aplican ambos descuentos, primero el del cliente y luego el cupón
function aplicarDescuentos(costo, cliente, cupon) {
```

```
  const conDescuentoCliente = costo - costo * porcentajeCliente(cliente);
  return conDescuentoCliente - conDescuentoCliente * porcentajeCupon(cupon);
}

function redondear(costo) {
  const exacto = Number(costo.toFixed(DECIMALES_TOLERANCIA));
  return Math.floor(exacto / MULTIPLO_REDONDEO + 0.5) * MULTIPLO_REDONDEO;
}

/**
 * Calcula el costo de un envío.
 * @param {number} peso   kg, 0 < peso <= 50
 * @param {string} zona   "urbana" | "rural" | "nacional" | "insular"
 * @param {string} cliente "normal" | "frecuente" | "vip"
 * @param {number} dia    1..7 (6 = sábado, 7 = domingo)
 * @param {string|null} [cupon] "ENVIO10" u omitido
 * @returns {number} costo en pesos, múltiplo de 100
 */
function calc(peso, zona, cliente, dia, cupon = null) {
  validarPeso(peso);
  validarDia(dia);
  const estrategiaZona = obtenerEstrategiaZona(zona);
  if (cupon !== null && cupon !== undefined && !estrategiaZona.aceptaCupon) {
    throw new Error("los envíos insulares no aceptan cupón");
  }
  let costo = estrategiaZona.aplicarRecargo(tarifaBase(peso));
  if (esFinDeSemana(dia)) costo += RECARGO_FIN_DE_SEMANA;
  costo = aplicarDescuentos(costo, cliente, cupon);
  return redondear(Math.max(costo, COSTO_MINIMO));
}

module.exports = {
  calc,
  calcularCosto: calc,
  tarifaBase,
  esFinDeSemana,
  aplicarDescuentos,
  redondear,
  EstrategiaZona,
  ZonaUrbana,
  ZonaConRecargo,
  ZonaRural,
  ZonaNacional,
  ZonaInsular,
  ESTRATEGIAS_ZONA,
};
```